

09 시계열





강의 내용

Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 Python 언어의 기본, lpython, 주피터 노트북	-
02주차	내장 자료구조, 함수, 파일	실습 과제
03주차	Numpy 기본 : 배열과 벡터 연산	실습 과제
04주차	Pandas 시작하기	실습 과제
05주차	데이터 로딩과 저장, 파일 형식	실습 과제
06주차	데이터 정제 및 준비(조인, 병합, 변형)	실습 과제
07주차	그래프와 시각화	기말 프로젝트 상세 공지
08주차	중간고사 - 오프라인 오픈북으로 진행	실습 과제

기간	내용	과제
09주차	데이터 집계와 그룹 연산	실습 과제
10주차	시계열	실습 과제
11주차	고급 pandas	실습 과제
12주차	파이썬 모델링 라이브러리	실습 과제
13주차	데이터 분석 예제 / 고급 Numpy	실습 과제
14주차	Kaggle / Dacon 데이터 분석 사례 Case Study	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!

1부: 시계열

- 1) 날짜 & 시간 데이터 타입 + 도구
- 2) 시계열 기초
- 3) 날짜 범위 & 빈도 & 시프팅
- 4) Time Zone 처리





시계열 데이터

- 금융, 경제학, 생태학, 신경과학, 물리학 등 다양한 분야에서 구조화된 데이터의 중요한 형태
- 여러 시점에서 관찰되거나 측정된 모든 것은 시계열을 형성
- 많은 시계열은 고정 빈도 기반으로 존재합니다.

즉, 데이터 포인트는 15초마다, 5분마다 또는 한 달에 한 번과 같은 일부 규칙에 따라 규칙적인 간격으로 발생합니다. 시계열은 고정된 시간 단위나 단위 간의 오프셋 없이 불규칙할 수도 있다



시계열 데이터를 표시하고 참조하는 방법은 애플리케이션에 따라 다르며 다음 중 하나가 있을 수 있다

- 타임스탬프, 특정 시점
- 고정 기간, 2007년 1월 또는 2010년 전체
- 시간 간격, 시작 및 종료 타임스탬프로 표시
마침표는 간격의 특별한 경우로 생각할 수 있습니다
- 실험 또는 경과 시간, 각 타임스탬프는 특정 시작 시간에 대한 시간 측정값(예: 오븐에 넣은 후 초당 굽는 쿠키의 직경)

이번 수업에서는 첫 세 범주의 시계열을 주로 활용하지만, 많은 기술이 실험 시작부터 경과 시간을 나타내는 정수 또는 부동 소수점 숫자가 될 수 있는 실험적 시계열에도 적용될 수 있다. 가장 간단하고 가장 널리 사용되는 시계열 종류는 타임스탬프로 인덱싱된 시계열!



pandas는 또한 실험 또는 경과 시간을 나타내는 유용한 방법이 될 수 있는 `timedelta`를 기반으로 하는 인덱스를 지원

(이번 수업 때는 `timedelta` 인덱스를 탐색하지 않지만 pandas 문서에서 더 많은 것을 배울 수 있음)

pandas는 많은 내장 시계열 도구와 데이터 알고리즘을 제공

- 매우 큰 시계열로 효율적으로 작업할 수 있고
- 불규칙 및 고정 주파수 시계열을 쉽게 슬라이스 및 다이스, 집계 및 리샘플링할 수 있음

이러한 도구 중 일부는 특히 금융 및 경제 응용 프로그램에 유용하지만, 서버 로그 데이터를 분석하는 데에도 사용할 수 있음



11.1 Date and Time Data Types and Tools

The Python standard library includes data types for date and time data, as well as calendar-related functionality. The `datetime`, `time`, and `calendar` modules are the main places to start. The `datetime.datetime` type, or simply `datetime`, is widely used:

```
In [10]: from datetime import datetime
```

```
In [11]: now = datetime.now()
```

```
In [12]: now
```

```
Out[12]: datetime.datetime(2017, 9, 25, 14, 5, 52, 72973)
```

```
In [13]: now.year, now.month, now.day
```

```
Out[13]: (2017, 9, 25)
```

`datetime` stores both the date and time down to the microsecond. `timedelta` represents the temporal difference between two `datetime` objects:

```
In [14]: delta = datetime(2011, 1, 7) - datetime(2008, 6, 24, 8, 15)
```

```
In [15]: delta
```

```
Out[15]: datetime.timedelta(926, 56700)
```

```
In [16]: delta.days
```

```
Out[16]: 926
```

```
In [17]: delta.seconds
```

```
Out[17]: 56700
```

You can add (or subtract) a `timedelta` or multiple thereof to a `datetime` object to yield a new shifted object:

```
In [18]: from datetime import timedelta
```

```
In [19]: start = datetime(2011, 1, 7)
```

```
In [20]: start + timedelta(12)
```

```
Out[20]: datetime.datetime(2011, 1, 19, 0, 0)
```

```
In [21]: start - 2 * timedelta(12)
```

```
Out[21]: datetime.datetime(2010, 12, 14, 0, 0)
```

Table 11-1 summarizes the data types in the `datetime` module. While this chapter is mainly concerned with the data types in `pandas` and higher-level time series manipulation, you may encounter the `datetime`-based types in many other places in Python in the wild.

Table 11-1. Types in `datetime` module

Type	Description
<code>date</code>	Store calendar date (year, month, day) using the Gregorian calendar
<code>time</code>	Store time of day as hours, minutes, seconds, and microseconds
<code>datetime</code>	Stores both date and time
<code>timedelta</code>	Represents the difference between two <code>datetime</code> values (as days, seconds, and microseconds)
<code>tzinfo</code>	Base type for storing time zone information



날짜 & 시간 데이터 타입 + 도구 > Converting Between String and Datetime

Date and Time Data Types and Tools

Converting Between String and Datetime

You can format datetime objects and pandas Timestamp objects, which I'll introduce later, as strings using `str` or the `strftime` method, passing a format specification:

```
In [22]: stamp = datetime(2011, 1, 3)
```

```
In [23]: str(stamp)
```

```
Out[23]: '2011-01-03 00:00:00'
```

```
In [24]: stamp.strftime('%Y-%m-%d')
```

```
Out[24]: '2011-01-03'
```

See [Table 11-2](#) for a complete list of the format codes (reproduced from [Chapter 2](#)).

Table 11-2. Datetime format specification (ISO C89 compatible)

Type	Description
%Y	Four-digit year
%y	Two-digit year
%m	Two-digit month [01, 12]
%d	Two-digit day [01, 31]
%H	Hour (24-hour clock) [00, 23]
%I	Hour (12-hour clock) [01, 12]
%M	Two-digit minute [00, 59]
%S	Second [00, 61] (seconds 60, 61 account for leap seconds)
%w	Weekday as integer [0 (Sunday), 6]

Type	Description
%U	Week number of the year [00, 53]; Sunday is considered the first day of the week, and days before the first Sunday of the year are "week 0"
%W	Week number of the year [00, 53]; Monday is considered the first day of the week, and days before the first Monday of the year are "week 0"
%z	UTC time zone offset as +HHMM or -HHMM; empty if time zone naive
%F	Shortcut for %Y-%m-%d (e.g., 2012-4-18)
%D	Shortcut for %m/%d/%y (e.g., 04/18/12)

You can use these same format codes to convert strings to dates using `datetime.strptime`:

```
In [25]: value = '2011-01-03'
```

```
In [26]: datetime.strptime(value, '%Y-%m-%d')
```

```
Out[26]: datetime.datetime(2011, 1, 3, 0, 0)
```

```
In [27]: datestrs = ['7/6/2011', '8/6/2011']
```

```
In [28]: [datetime.strptime(x, '%m/%d/%Y') for x in datestrs]
```

```
Out[28]:
```

```
[datetime.datetime(2011, 7, 6, 0, 0),
```

```
datetime.datetime(2011, 8, 6, 0, 0)]
```




날짜 & 시간 데이터 타입 + 도구 > *Converting Between String and Datetime*

Date and Time Data Types and Tools

`datetime.strptime` is a good way to parse a date with a known format. However, it can be a bit annoying to have to write a format spec each time, especially for common date formats. In this case, you can use the `parser.parse` method in the third-party `dateutil` package (this is installed automatically when you install pandas):

```
In [29]: from dateutil.parser import parse

In [30]: parse('2011-01-03')
Out[30]: datetime.datetime(2011, 1, 3, 0, 0)
```

`dateutil` is capable of parsing most human-intelligible date representations:

```
In [31]: parse('Jan 31, 1997 10:45 PM')
Out[31]: datetime.datetime(1997, 1, 31, 22, 45)
```

In international locales, day appearing before month is very common, so you can pass `dayfirst=True` to indicate this:

```
In [32]: parse('6/12/2011', dayfirst=True)
Out[32]: datetime.datetime(2011, 12, 6, 0, 0)
```

pandas is generally oriented toward working with arrays of dates, whether used as an axis index or a column in a DataFrame. The `to_datetime` method parses many different kinds of date representations. Standard date formats like ISO 8601 can be parsed very quickly:

```
In [33]: datestrs = ['2011-07-06 12:00:00', '2011-08-06 00:00:00']

In [34]: pd.to_datetime(datestrs)
Out[34]: DatetimeIndex(['2011-07-06 12:00:00', '2011-08-06 00:00:00'], dtype='datetime64[ns]', freq=None)
```

It also handles values that should be considered missing (None, empty string, etc.):

```
In [35]: idx = pd.to_datetime(datestrs + [None])

In [36]: idx
Out[36]: DatetimeIndex(['2011-07-06 12:00:00', '2011-08-06 00:00:00', 'NaT'], dtype='datetime64[ns]', freq=None)

In [37]: idx[2]
Out[37]: NaT

In [38]: pd.isnull(idx)
Out[38]: array([False, False,  True], dtype=bool)
```

NaT (Not a Time) is pandas's null value for timestamp data.



날짜 & 시간 데이터 타입 + 도구 > Converting Between String and Datetime

Date and Time Data Types and Tools



`dateutil.parser` is a useful but imperfect tool. Notably, it will recognize some strings as dates that you might prefer that it didn't—for example, `'42'` will be parsed as the year 2042 with today's calendar date.

`datetime` objects also have a number of locale-specific formatting options for systems in other countries or languages. For example, the abbreviated month names will be different on German or French systems compared with English systems. See [Table 11-3](#) for a listing.

Table 11-3. Locale-specific date formatting

Type	Description
%a	Abbreviated weekday name
%A	Full weekday name
%b	Abbreviated month name
%B	Full month name
%c	Full date and time (e.g., 'Tue 01 May 2012 04:20:57 PM')
%p	Locale equivalent of AM or PM
%x	Locale-appropriate formatted date (e.g., in the United States, May 1, 2012 yields '05/01/2012')
%X	Locale-appropriate time (e.g., '04:24:12 PM')



11.2 Time Series Basics

A basic kind of time series object in pandas is a Series indexed by timestamps, which is often represented external to pandas as Python strings or datetime objects:

```
In [39]: from datetime import datetime

In [40]: dates = [datetime(2011, 1, 2), datetime(2011, 1, 5),
....:             datetime(2011, 1, 7), datetime(2011, 1, 8),
....:             datetime(2011, 1, 10), datetime(2011, 1, 12)]

In [41]: ts = pd.Series(np.random.randn(6), index=dates)

In [42]: ts
Out[42]:
2011-01-02    -0.204708
2011-01-05     0.478943
2011-01-07    -0.519439
2011-01-08    -0.555730
2011-01-10     1.965781
2011-01-12     1.393406
dtype: float64
```

Under the hood, these datetime objects have been put in a DatetimeIndex:

```
In [43]: ts.index
Out[43]:
DatetimeIndex(['2011-01-02', '2011-01-05', '2011-01-07', '2011-01-08',
              '2011-01-10', '2011-01-12'],
              dtype='datetime64[ns]', freq=None)
```

Like other Series, arithmetic operations between differently indexed time series automatically align on the dates:

```
In [44]: ts + ts[::-2]
Out[44]:
2011-01-02    -0.409415
2011-01-05         NaN
2011-01-07    -1.038877
2011-01-08         NaN
2011-01-10     3.931561
2011-01-12         NaN
dtype: float64
```

Recall that `ts[::-2]` selects every second element in `ts`.

pandas stores timestamps using NumPy's `datetime64` data type at the nanosecond resolution:

```
In [45]: ts.index.dtype
Out[45]: dtype('<M8[ns]')
```

Scalar values from a DatetimeIndex are pandas Timestamp objects:

```
In [46]: stamp = ts.index[0]

In [47]: stamp
Out[47]: Timestamp('2011-01-02 00:00:00')
```

A Timestamp can be substituted anywhere you would use a datetime object. Additionally, it can store frequency information (if any) and understands how to do time zone conversions and other kinds of manipulations. More on both of these things later.



시계열 기초 > Indexing, Selection, Subsetting

Time Series Basics

Indexing, Selection, Subsetting

Time series behaves like any other pandas.Series when you are indexing and selecting data based on label:

```
In [48]: stamp = ts.index[2]
```

```
In [49]: ts[stamp]
```

```
Out[49]: -0.51943871505673811
```

As a convenience, you can also pass a string that is interpretable as a date:

```
In [50]: ts['1/10/2011']
```

```
Out[50]: 1.9657805725027142
```

```
In [51]: ts['20110110']
```

```
Out[51]: 1.9657805725027142
```

For longer time series, a year or only a year and month can be passed to easily select slices of data:

```
In [52]: longer_ts = pd.Series(np.random.randn(1000),  
.....:                        index=pd.date_range('1/1/2000', periods=1000))
```

```
In [53]: longer_ts
```

```
Out[53]:
```

```
2000-01-01    0.092908
```

```
2000-01-02    0.281746
```

```
2000-01-03    0.769023
```

```
2000-01-04    1.246435
```

```
2000-01-05    1.007189
```

```
2000-01-06   -1.296221
```

```
2000-01-07    0.274992
```

```
2000-01-08    0.228913
```

```
2000-01-09    1.352917
```

```
2000-01-10    0.886429
```

```
...
```

```
2002-09-17   -0.139298
```

```
2002-09-18   -1.159926
```

```
2002-09-19    0.618965
```

```
2002-09-20    1.373890
```

```
2002-09-21   -0.983505
```

```
2002-09-22    0.930944
```

```
2002-09-23   -0.811676
```

```
2002-09-24   -1.830156
```

```
2002-09-25   -0.138730
```

```
2002-09-26    0.334088
```

```
Freq: D, Length: 1000, dtype: float64
```



시계열 기초 > Indexing, Selection, Subsetting

Time Series Basics

```
In [54]: longer_ts['2001']
```

```
Out[54]:
```

```
2001-01-01    1.599534
2001-01-02    0.474071
2001-01-03    0.151326
2001-01-04   -0.542173
2001-01-05   -0.475496
2001-01-06    0.106403
2001-01-07   -1.308228
2001-01-08    2.173185
2001-01-09    0.564561
2001-01-10   -0.190481
```

```
2001-12-22    0.000369
2001-12-23    0.900885
2001-12-24   -0.454869
2001-12-25   -0.864547
2001-12-26    1.129120
2001-12-27    0.057874
2001-12-28   -0.433739
2001-12-29    0.092698
2001-12-30   -1.397820
2001-12-31    1.457823
```

```
Freq: D, Length: 365, dtype: float64
```

Here, the string '2001' is interpreted as a year and selects that time period. This also works if you specify the month:

```
In [55]: longer_ts['2001-05']
```

```
Out[55]:
```

```
2001-05-01   -0.622547
2001-05-02    0.936289
2001-05-03    0.750018
2001-05-04   -0.056715
2001-05-05    2.300675
2001-05-06    0.569497
2001-05-07    1.489410
2001-05-08    1.264250
2001-05-09   -0.761837
2001-05-10   -0.331617
```

```
2001-05-22    0.503699
2001-05-23   -1.387874
2001-05-24    0.204851
2001-05-25    0.603705
2001-05-26    0.545680
2001-05-27    0.235477
2001-05-28    0.111835
2001-05-29   -1.251504
2001-05-30   -2.949343
2001-05-31    0.634634
```

```
Freq: D, Length: 31, dtype: float64
```



시계열 기초 > Indexing, Selection, Subsetting

Time Series Basics

Slicing with `datetime` objects works as well:

```
In [56]: ts[datetime(2011, 1, 7):]
Out[56]:
2011-01-07    -0.519439
2011-01-08    -0.555730
2011-01-10     1.965781
2011-01-12     1.393406
dtype: float64
```

Because most time series data is ordered chronologically, you can slice with timestamps not contained in a time series to perform a range query:

```
In [57]: ts
Out[57]:
2011-01-02    -0.204708
2011-01-05     0.478943
2011-01-07    -0.519439
2011-01-08    -0.555730
2011-01-10     1.965781
2011-01-12     1.393406
dtype: float64
```

```
In [58]: ts['1/6/2011':'1/11/2011']
Out[58]:
2011-01-07    -0.519439
2011-01-08    -0.555730
2011-01-10     1.965781
dtype: float64
```

As before, you can pass either a string date, `datetime`, or timestamp. Remember that slicing in this manner produces views on the source time series like slicing NumPy arrays. This means that no data is copied and modifications on the slice will be reflected in the original data.

There is an equivalent instance method, `truncate`, that slices a Series between two dates:

```
In [59]: ts.truncate(after='1/9/2011')
Out[59]:
2011-01-02    -0.204708
2011-01-05     0.478943
2011-01-07    -0.519439
2011-01-08    -0.555730
dtype: float64
```

All of this holds true for `DataFrame` as well, indexing on its rows:

```
In [60]: dates = pd.date_range('1/1/2000', periods=100, freq='W-WED')

In [61]: long_df = pd.DataFrame(np.random.randn(100, 4),
....:                           index=dates,
....:                           columns=['Colorado', 'Texas',
....:                                   'New York', 'Ohio'])
```

```
In [62]: long_df.loc['5-2001']
Out[62]:
```

	Colorado	Texas	New York	Ohio
2001-05-02	-0.006045	0.490094	-0.277186	-0.707213
2001-05-09	-0.560107	2.735527	0.927335	1.513906
2001-05-16	0.538600	1.273768	0.667876	-0.969206
2001-05-23	1.676091	-0.817649	0.050188	1.951312
2001-05-30	3.260383	0.963301	1.201206	-1.852001

Time Series with Duplicate Indices

In some applications, there may be multiple data observations falling on a particular timestamp. Here is an example:

```
In [63]: dates = pd.DatetimeIndex(['1/1/2000', '1/2/2000', '1/2/2000',  
.....:                           '1/2/2000', '1/3/2000'])
```

```
In [64]: dup_ts = pd.Series(np.arange(5), index=dates)
```

```
In [65]: dup_ts
```

```
Out[65]:
```

```
2000-01-01    0
```

```
2000-01-02    1
```

```
2000-01-02    2
```

```
2000-01-02    3
```

```
2000-01-03    4
```

```
dtype: int64
```

We can tell that the index is not unique by checking its `is_unique` property:

```
In [66]: dup_ts.index.is_unique
```

```
Out[66]: False
```

Indexing into this time series will now either produce scalar values or slices depending on whether a timestamp is duplicated:

```
In [67]: dup_ts['1/3/2000'] # not duplicated
```

```
Out[67]: 4
```

```
In [68]: dup_ts['1/2/2000'] # duplicated
```

```
Out[68]:
```

```
2000-01-02    1
```

```
2000-01-02    2
```

```
2000-01-02    3
```

```
dtype: int64
```

Suppose you wanted to aggregate the data having non-unique timestamps. One way to do this is to use `groupby` and pass `level=0`:

```
In [69]: grouped = dup_ts.groupby(level=0)
```

```
In [70]: grouped.mean()
```

```
Out[70]:
```

```
2000-01-01    0
```

```
2000-01-02    2
```

```
2000-01-03    4
```

```
dtype: int64
```

```
In [71]: grouped.count()
```

```
Out[71]:
```

```
2000-01-01    1
```

```
2000-01-02    3
```

```
2000-01-03    1
```

```
dtype: int64
```




11.3 Date Ranges, Frequencies, and Shifting

Generic time series in pandas are assumed to be irregular; that is, they have no fixed frequency. For many applications this is sufficient. However, it's often desirable to work relative to a fixed frequency, such as daily, monthly, or every 15 minutes, even if that means introducing missing values into a time series. Fortunately pandas has a full suite of standard time series frequencies and tools for resampling, inferring frequencies, and generating fixed-frequency date ranges. For example, you can convert the sample time series to be fixed daily frequency by calling `resample`:

```
In [72]: ts
Out[72]:
2011-01-02    -0.204708
2011-01-05     0.478943
2011-01-07    -0.519439
2011-01-08    -0.555730
2011-01-10     1.965781
2011-01-12     1.393406
dtype: float64
```

```
In [73]: resampler = ts.resample('D')
```

The string 'D' is interpreted as daily frequency.

Conversion between frequencies or *resampling* is a big enough topic to have its own section later ([Section 11.6, “Resampling and Frequency Conversion,” on page 348](#)). Here I'll show you how to use the base frequencies and multiples thereof.



날짜 범위 & 빈도 & 시프팅 > *Generating Date Ranges*

Date Ranges, Frequencies, and Shifting

Generating Date Ranges

While I used it previously without explanation, `pandas.date_range` is responsible for generating a `DatetimeIndex` with an indicated length according to a particular frequency:

```
In [74]: index = pd.date_range('2012-04-01', '2012-06-01')
```

```
In [75]: index
```

```
Out[75]:
```

```
DatetimeIndex(['2012-04-01', '2012-04-02', '2012-04-03', '2012-04-04',  
              '2012-04-05', '2012-04-06', '2012-04-07', '2012-04-08',  
              '2012-04-09', '2012-04-10', '2012-04-11', '2012-04-12',  
              '2012-04-13', '2012-04-14', '2012-04-15', '2012-04-16',  
              '2012-04-17', '2012-04-18', '2012-04-19', '2012-04-20',  
              '2012-04-21', '2012-04-22', '2012-04-23', '2012-04-24',  
              '2012-04-25', '2012-04-26', '2012-04-27', '2012-04-28',  
              '2012-04-29', '2012-04-30', '2012-05-01', '2012-05-02',  
              '2012-05-03', '2012-05-04', '2012-05-05', '2012-05-06',  
              '2012-05-07', '2012-05-08', '2012-05-09', '2012-05-10',  
              '2012-05-11', '2012-05-12', '2012-05-13', '2012-05-14',  
              '2012-05-15', '2012-05-16', '2012-05-17', '2012-05-18',  
              '2012-05-19', '2012-05-20', '2012-05-21', '2012-05-22',  
              '2012-05-23', '2012-05-24', '2012-05-25', '2012-05-26',  
              '2012-05-27', '2012-05-28', '2012-05-29', '2012-05-30',  
              '2012-05-31', '2012-06-01'],  
              dtype='datetime64[ns]', freq='D')
```

By default, `date_range` generates daily timestamps. If you pass only a start or end date, you must pass a number of periods to generate:

```
In [76]: pd.date_range(start='2012-04-01', periods=20)
```

```
Out[76]:
```

```
DatetimeIndex(['2012-04-01', '2012-04-02', '2012-04-03', '2012-04-04',  
              '2012-04-05', '2012-04-06', '2012-04-07', '2012-04-08',  
              '2012-04-09', '2012-04-10', '2012-04-11', '2012-04-12',  
              '2012-04-13', '2012-04-14', '2012-04-15', '2012-04-16',  
              '2012-04-17', '2012-04-18', '2012-04-19', '2012-04-20'],  
              dtype='datetime64[ns]', freq='D')
```

```
In [77]: pd.date_range(end='2012-06-01', periods=20)
```

```
Out[77]:
```

```
DatetimeIndex(['2012-05-13', '2012-05-14', '2012-05-15', '2012-05-16',  
              '2012-05-17', '2012-05-18', '2012-05-19', '2012-05-20',  
              '2012-05-21', '2012-05-22', '2012-05-23', '2012-05-24',  
              '2012-05-25', '2012-05-26', '2012-05-27', '2012-05-28',  
              '2012-05-29', '2012-05-30', '2012-05-31', '2012-06-01'],  
              dtype='datetime64[ns]', freq='D')
```



날짜 범위 & 빈도 & 시프팅 > Generating Date Ranges

Date Ranges, Frequencies, and Shifting

The start and end dates define strict boundaries for the generated date index. For example, if you wanted a date index containing the last business day of each month, you would pass the 'BM' frequency (business end of month; see more complete listing of frequencies in Table 11-4) and only dates falling on or inside the date interval will be included:

```
In [78]: pd.date_range('2000-01-01', '2000-12-01', freq='BM')
Out[78]:
DatetimeIndex(['2000-01-31', '2000-02-29', '2000-03-31', '2000-04-28',
                '2000-05-31', '2000-06-30', '2000-07-31', '2000-08-31',
                '2000-09-29', '2000-10-31', '2000-11-30'],
              dtype='datetime64[ns]', freq='BM')
```

Table 11-4. Base time series frequencies (not comprehensive)

Alias	Offset type	Description
D	Day	Calendar daily
B	BusinessDay	Business daily
H	Hour	Hourly
T or min	Minute	Minutely
S	Second	Secondly
L or ms	Milli	Millisecond (1/1,000 of 1 second)
U	Micro	Microsecond (1/1,000,000 of 1 second)
M	MonthEnd	Last calendar day of month
BM	BusinessMonthEnd	Last business day (weekday) of month
MS	MonthBegin	First calendar day of month
BMS	BusinessMonthBegin	First weekday of month
W-MON, W-TUE, ...	Week	Weekly on given day of week (MON, TUE, WED, THU, FRI, SAT, or SUN)

Alias	Offset type	Description
WOM-1MON, WOM-2MON, ...	WeekOfMonth	Generate weekly dates in the first, second, third, or fourth week of the month (e.g., WOM-3FRI for the third Friday of each month)
Q-JAN, Q-FEB, ...	QuarterEnd	Quarterly dates anchored on last calendar day of each month, for year ending in indicated month (JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, or DEC)
BQ-JAN, BQ-FEB, ...	BusinessQuarterEnd	Quarterly dates anchored on last weekday day of each month, for year ending in indicated month
QS-JAN, QS-FEB, ...	QuarterBegin	Quarterly dates anchored on first calendar day of each month, for year ending in indicated month
BQS-JAN, BQS-FEB, ...	BusinessQuarterBegin	Quarterly dates anchored on first weekday day of each month, for year ending in indicated month
A-JAN, A-FEB, ...	YearEnd	Annual dates anchored on last calendar day of given month (JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, or DEC)
BA-JAN, BA-FEB, ...	BusinessYearEnd	Annual dates anchored on last weekday of given month
AS-JAN, AS-FEB, ...	YearBegin	Annual dates anchored on first day of given month
BAS-JAN, BAS-FEB, ...	BusinessYearBegin	Annual dates anchored on first weekday of given month

date_range by default preserves the time (if any) of the start or end timestamp:

```
In [79]: pd.date_range('2012-05-02 12:56:31', periods=5)
Out[79]:
DatetimeIndex(['2012-05-02 12:56:31', '2012-05-03 12:56:31',
                '2012-05-04 12:56:31', '2012-05-05 12:56:31',
                '2012-05-06 12:56:31'],
              dtype='datetime64[ns]', freq='D')
```

Sometimes you will have start or end dates with time information but want to generate a set of timestamps *normalized* to midnight as a convention. To do this, there is a `normalize` option:

```
In [80]: pd.date_range('2012-05-02 12:56:31', periods=5, normalize=True)
Out[80]:
DatetimeIndex(['2012-05-02', '2012-05-03', '2012-05-04', '2012-05-05',
                '2012-05-06'],
              dtype='datetime64[ns]', freq='D')
```



날짜 범위 & 빈도 & 시프팅 > Frequencies and Date Offsets

Date Ranges, Frequencies, and Shifting

Frequencies and Date Offsets

Frequencies in pandas are composed of a *base frequency* and a multiplier. Base frequencies are typically referred to by a string alias, like 'M' for monthly or 'H' for hourly. For each base frequency, there is an object defined generally referred to as a *date offset*. For example, hourly frequency can be represented with the Hour class:

```
In [81]: from pandas.tseries.offsets import Hour, Minute
```

```
In [82]: hour = Hour()
```

```
In [83]: hour
```

```
Out[83]: <Hour>
```

You can define a multiple of an offset by passing an integer:

```
In [84]: four_hours = Hour(4)
```

```
In [85]: four_hours
```

```
Out[85]: <4 * Hours>
```

In most applications, you would never need to explicitly create one of these objects, instead using a string alias like 'H' or '4H'. Putting an integer before the base frequency creates a multiple:

```
In [86]: pd.date_range('2000-01-01', '2000-01-03 23:59', freq='4h')
Out[86]:
DatetimeIndex(['2000-01-01 00:00:00', '2000-01-01 04:00:00',
               '2000-01-01 08:00:00', '2000-01-01 12:00:00',
               '2000-01-01 16:00:00', '2000-01-01 20:00:00',
               '2000-01-02 00:00:00', '2000-01-02 04:00:00',
               '2000-01-02 08:00:00', '2000-01-02 12:00:00',
               '2000-01-02 16:00:00', '2000-01-02 20:00:00',
               '2000-01-03 00:00:00', '2000-01-03 04:00:00',
               '2000-01-03 08:00:00', '2000-01-03 12:00:00',
               '2000-01-03 16:00:00', '2000-01-03 20:00:00'],
              dtype='datetime64[ns]', freq='4H')
```

Many offsets can be combined together by addition:

```
In [87]: Hour(2) + Minute(30)
Out[87]: <150 * Minutes>
```

Similarly, you can pass frequency strings, like '1h30min', that will effectively be parsed to the same expression:

```
In [88]: pd.date_range('2000-01-01', periods=10, freq='1h30min')
Out[88]:
DatetimeIndex(['2000-01-01 00:00:00', '2000-01-01 01:30:00',
               '2000-01-01 03:00:00', '2000-01-01 04:30:00',
               '2000-01-01 06:00:00', '2000-01-01 07:30:00',
               '2000-01-01 09:00:00', '2000-01-01 10:30:00',
               '2000-01-01 12:00:00', '2000-01-01 13:30:00'],
              dtype='datetime64[ns]', freq='90T')
```




날짜 범위 & 빈도 & 시프팅 > Frequencies and Date Offsets

Date Ranges, Frequencies, and Shifting

Some frequencies describe points in time that are not evenly spaced. For example, 'M' (calendar month end) and 'BM' (last business/weekday of month) depend on the number of days in a month and, in the latter case, whether the month ends on a weekend or not. We refer to these as *anchored* offsets.

Refer back to **Table 11-4** for a listing of frequency codes and date offset classes available in pandas.



Users can define their own custom frequency classes to provide date logic not available in pandas, though the full details of that are outside the scope of this book.

Week of month dates

One useful frequency class is “week of month,” starting with WOM. This enables you to get dates like the third Friday of each month:

```
In [89]: rng = pd.date_range('2012-01-01', '2012-09-01', freq='WOM-3FRI')
```

```
In [90]: list(rng)
```

```
Out[90]:
```

```
[Timestamp('2012-01-20 00:00:00', freq='WOM-3FRI'),  
 Timestamp('2012-02-17 00:00:00', freq='WOM-3FRI'),  
 Timestamp('2012-03-16 00:00:00', freq='WOM-3FRI'),  
 Timestamp('2012-04-20 00:00:00', freq='WOM-3FRI'),  
 Timestamp('2012-05-18 00:00:00', freq='WOM-3FRI'),  
 Timestamp('2012-06-15 00:00:00', freq='WOM-3FRI'),  
 Timestamp('2012-07-20 00:00:00', freq='WOM-3FRI'),  
 Timestamp('2012-08-17 00:00:00', freq='WOM-3FRI')]
```



날짜 범위 & 빈도 & 시프팅 > Shifting (Leading and Lagging) Data

Date Ranges, Frequencies, and Shifting

선행 지표와 후행 지표

선행 지표는 우리가 바라는 성과와 인과적으로 연결된 상황들을 보고하는 지표입니다. 그러므로 선행 지표는 어떠한 조치를 제안할 수 있는 것을 측정합니다. 이에 반해, 후행 지표는 바뀔 수 없는 상황들을 보고합니다.

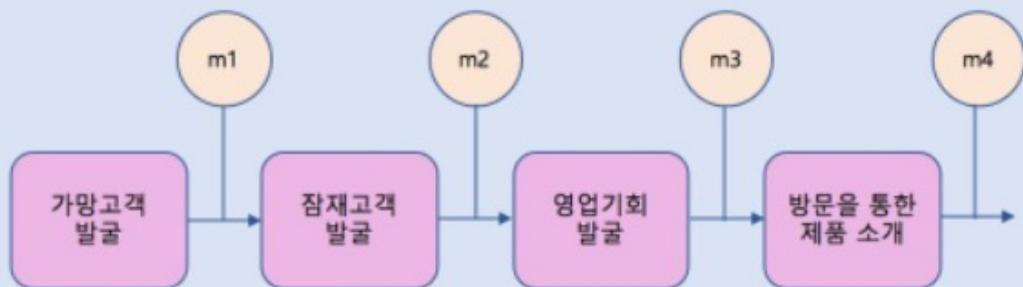
예를 들어, 빔 프로젝터 제조업체의 판매 관리자가 매 분기마다 15%의 판매 증가라는 목표를 수립했다고 가정하는 경우, 이 관리자는 매 분기 말까지 기다린 후에야 얼마나 많은 빔 프로젝터가 판매되었는지 확인할 수 있습니다. 그러나 이러한 후행 지표로는 목표 달성 여부를 판단할 수 있지만 달성된 결과 자체를 바꿀 수는 없습니다.

해당 관리자는 빔 프로젝터 판매 데이터를 추적하여 가망 고객의 약 10%가 잠재 고객이 되고, 영업직원들은 50% 잠재 고객에 대해 방문 약속을 잡으며 그 중 약 20%가 빔 프로젝터를 구매한다는 사실을 알게 되었습니다.

만약 영업직원들이 이번 분기에 100번의 고객 방문을 계획하고 있다면 관리자는 대략 20번의 판매가 이루어질 것임을 예측할 수 있습니다. 그러므로 고객 방문 회수(m3)는 선행지표가 되고 판매 회수(m4)는 후행 지표가 됩니다. 이 사례에서 가장 좋은 방안은 선행 지표인 가망 고객의 수(m1)를 추적하는 것이며, 계산해보면 100명의 가망 고객에 대해 대략 1건의 판매($=100 \times 0.1 \times 0.5 \times 0.2$)가 이루어짐을 알 수 있습니다.

이와 같이 선행 지표를 추적, 관리함으로써 기업의 목표 달성에 크게 기여할 수 있습니다.

전략적 목표: 매 분기마다 15%의 판매 증가



M1 - 분기별 가망고객 수

M2 - 분기별 잠재고객 수

M3 - 분기별 고객 방문 회수

M4 - 분기별 판매 회수

<https://www.puzzledata.com/blog171020/>



날짜 범위 & 빈도 & 시프팅 > Shifting (Leading and Lagging) Data

Date Ranges, Frequencies, and Shifting

Shifting (Leading and Lagging) Data

“Shifting” refers to moving data backward and forward through time. Both Series and DataFrame have a `shift` method for doing naive shifts forward or backward, leaving the index unmodified:

```
In [91]: ts = pd.Series(np.random.randn(4),
      ....:               index=pd.date_range('1/1/2000', periods=4, freq='M'))
```

```
In [92]: ts
Out[92]:
2000-01-31    -0.066748
2000-02-29     0.838639
2000-03-31    -0.117388
2000-04-30    -0.517795
Freq: M, dtype: float64
```

```
In [93]: ts.shift(2)
Out[93]:
2000-01-31         NaN
2000-02-29         NaN
2000-03-31    -0.066748
2000-04-30     0.838639
Freq: M, dtype: float64
```

```
In [94]: ts.shift(-2)
Out[94]:
2000-01-31    -0.117388
2000-02-29    -0.517795
2000-03-31         NaN
2000-04-30         NaN
Freq: M, dtype: float64
```

When we shift like this, missing data is introduced either at the start or the end of the time series.

A common use of `shift` is computing percent changes in a time series or multiple time series as DataFrame columns. This is expressed as:

```
ts / ts.shift(1) - 1
```

Because naive shifts leave the index unmodified, some data is discarded. Thus if the frequency is known, it can be passed to `shift` to advance the timestamps instead of simply the data:

```
In [95]: ts.shift(2, freq='M')
Out[95]:
2000-03-31    -0.066748
2000-04-30     0.838639
2000-05-31    -0.117388
2000-06-30    -0.517795
Freq: M, dtype: float64
```

Other frequencies can be passed, too, giving you some flexibility in how to lead and lag the data:

```
In [96]: ts.shift(3, freq='D')
Out[96]:
2000-02-03    -0.066748
2000-03-03     0.838639
2000-04-03    -0.117388
2000-05-03    -0.517795
dtype: float64
```

```
In [97]: ts.shift(1, freq='90T')
Out[97]:
2000-01-31 01:30:00    -0.066748
2000-02-29 01:30:00     0.838639
2000-03-31 01:30:00    -0.117388
2000-04-30 01:30:00    -0.517795
Freq: M, dtype: float64
```

The T here stands for minutes.



날짜 범위 & 빈도 & 시프팅 > *Shifting (Leading and Lagging) Data*

Date Ranges, Frequencies, and Shifting

Shifting dates with offsets

The pandas date offsets can also be used with datetime or Timestamp objects:

```
In [98]: from pandas.tseries.offsets import Day, MonthEnd
```

```
In [99]: now = datetime(2011, 11, 17)
```

```
In [100]: now + 3 * Day()
Out[100]: Timestamp('2011-11-20 00:00:00')
```

If you add an anchored offset like MonthEnd, the first increment will “roll forward” a date to the next date according to the frequency rule:

```
In [101]: now + MonthEnd()
Out[101]: Timestamp('2011-11-30 00:00:00')
```

```
In [102]: now + MonthEnd(2)
Out[102]: Timestamp('2011-12-31 00:00:00')
```

Anchored offsets can explicitly “roll” dates forward or backward by simply using their rollforward and rollback methods, respectively:

```
In [103]: offset = MonthEnd()

In [104]: offset.rollforward(now)
Out[104]: Timestamp('2011-11-30 00:00:00')

In [105]: offset.rollback(now)
Out[105]: Timestamp('2011-10-31 00:00:00')
```

A creative use of date offsets is to use these methods with groupby:

```
In [106]: ts = pd.Series(np.random.randn(20),
.....:                  index=pd.date_range('1/15/2000', periods=20, freq='4d'))
```

```
In [107]: ts
Out[107]:
2000-01-15    -0.116696
2000-01-19     2.389645
2000-01-23    -0.932454
2000-01-27    -0.229331
2000-01-31    -1.140330
2000-02-04     0.439920
2000-02-08    -0.823758
2000-02-12    -0.520930
2000-02-16     0.350282
2000-02-20     0.204395
2000-02-24     0.133445
2000-02-28     0.327905
2000-03-03     0.072153
2000-03-07     0.131678
2000-03-11    -1.297459
2000-03-15     0.997747
2000-03-19     0.870955
2000-03-23    -0.991253
2000-03-27     0.151699
2000-03-31     1.266151
Freq: 4D, dtype: float64
```

```
In [108]: ts.groupby(offset.rollforward).mean()
Out[108]:
2000-01-31    -0.005833
2000-02-29     0.015894
2000-03-31     0.150209
dtype: float64
```

Of course, an easier and faster way to do this is using resample (we’ll discuss this in much more depth in Section 11.6, “Resampling and Frequency Conversion,” on page 348):

```
In [109]: ts.resample('M').mean()
Out[109]:
2000-01-31    -0.005833
2000-02-29     0.015894
2000-03-31     0.150209
Freq: M, dtype: float64
```



11.4 Time Zone Handling

Working with time zones is generally considered one of the most unpleasant parts of time series manipulation. As a result, many time series users choose to work with time series in *coordinated universal time* or *UTC*, which is the successor to Greenwich Mean Time and is the current international standard. Time zones are expressed as offsets from UTC; for example, New York is four hours behind UTC during daylight saving time and five hours behind the rest of the year.

In Python, time zone information comes from the third-party `pytz` library (installable with `pip` or `conda`), which exposes the *Olson database*, a compilation of world time zone information. This is especially important for historical data because the daylight saving time (DST) transition dates (and even UTC offsets) have been changed numerous times depending on the whims of local governments. In the United States, the DST transition times have been changed many times since 1900!

For detailed information about the `pytz` library, you'll need to look at that library's documentation. As far as this book is concerned, `pandas` wraps `pytz`'s functionality so you can ignore its API outside of the time zone names. Time zone names can be found interactively and in the docs:

```
In [110]: import pytz
```

```
In [111]: pytz.common_timezones[-5:]
```

```
Out[111]: ['US/Eastern', 'US/Hawaii', 'US/Mountain', 'US/Pacific', 'UTC']
```

To get a time zone object from `pytz`, use `pytz.timezone`:

```
In [112]: tz = pytz.timezone('America/New_York')
```

```
In [113]: tz
```

```
Out[113]: <DstTzInfo 'America/New_York' LMT-1 day, 19:04:00 STD>
```

Methods in `pandas` will accept either time zone names or these objects.



Time Zone 처리 > Time Zone Localization and Conversion

Time Zone Handling

Time Zone Localization and Conversion

By default, time series in pandas are *time zone naive*. For example, consider the following time series:

```
In [114]: rng = pd.date_range('3/9/2012 9:30', periods=6, freq='D')
```

```
In [115]: ts = pd.Series(np.random.randn(len(rng)), index=rng)
```

```
In [116]: ts
```

```
Out[116]:
2012-03-09 09:30:00    -0.202469
2012-03-10 09:30:00     0.050718
2012-03-11 09:30:00     0.639869
2012-03-12 09:30:00     0.597594
2012-03-13 09:30:00    -0.797246
2012-03-14 09:30:00     0.472879
Freq: D, dtype: float64
```

The index's tz field is None:

```
In [117]: print(ts.index.tz)
None
```

Date ranges can be generated with a time zone set:

```
In [118]: pd.date_range('3/9/2012 9:30', periods=10, freq='D', tz='UTC')
```

```
Out[118]:
DatetimeIndex(['2012-03-09 09:30:00+00:00', '2012-03-10 09:30:00+00:00',
               '2012-03-11 09:30:00+00:00', '2012-03-12 09:30:00+00:00',
               '2012-03-13 09:30:00+00:00', '2012-03-14 09:30:00+00:00',
               '2012-03-15 09:30:00+00:00', '2012-03-16 09:30:00+00:00',
               '2012-03-17 09:30:00+00:00', '2012-03-18 09:30:00+00:00'],
              dtype='datetime64[ns, UTC]', freq='D')
```

Conversion from naive to *localized* is handled by the `tz_localize` method:

```
In [119]: ts
Out[119]:
2012-03-09 09:30:00    -0.202469
2012-03-10 09:30:00     0.050718
2012-03-11 09:30:00     0.639869
2012-03-12 09:30:00     0.597594
2012-03-13 09:30:00    -0.797246
2012-03-14 09:30:00     0.472879
Freq: D, dtype: float64
```

```
In [120]: ts_utc = ts.tz_localize('UTC')
```

```
In [121]: ts_utc
Out[121]:
2012-03-09 09:30:00+00:00    -0.202469
2012-03-10 09:30:00+00:00     0.050718
2012-03-11 09:30:00+00:00     0.639869
2012-03-12 09:30:00+00:00     0.597594
2012-03-13 09:30:00+00:00    -0.797246
2012-03-14 09:30:00+00:00     0.472879
Freq: D, dtype: float64
```

```
In [122]: ts_utc.index
Out[122]:
DatetimeIndex(['2012-03-09 09:30:00+00:00', '2012-03-10 09:30:00+00:00',
               '2012-03-11 09:30:00+00:00', '2012-03-12 09:30:00+00:00',
               '2012-03-13 09:30:00+00:00', '2012-03-14 09:30:00+00:00'],
              dtype='datetime64[ns, UTC]', freq='D')
```




Time Zone 처리 > Time Zone Localization and Conversion

Time Zone Handling

Once a time series has been localized to a particular time zone, it can be converted to another time zone with `tz_convert`:

```
In [123]: ts_utc.tz_convert('America/New_York')
Out[123]:
2012-03-09 04:30:00-05:00    -0.202469
2012-03-10 04:30:00-05:00     0.050718
2012-03-11 05:30:00-04:00     0.639869
2012-03-12 05:30:00-04:00     0.597594
2012-03-13 05:30:00-04:00    -0.797246
2012-03-14 05:30:00-04:00     0.472879
Freq: D, dtype: float64
```

In the case of the preceding time series, which straddles a DST transition in the America/New_York time zone, we could localize to EST and convert to, say, UTC or Berlin time:

```
In [124]: ts_eastern = ts.tz_localize('America/New_York')
```

```
In [125]: ts_eastern.tz_convert('UTC')
Out[125]:
2012-03-09 14:30:00+00:00    -0.202469
2012-03-10 14:30:00+00:00     0.050718
2012-03-11 13:30:00+00:00     0.639869
2012-03-12 13:30:00+00:00     0.597594
2012-03-13 13:30:00+00:00    -0.797246
2012-03-14 13:30:00+00:00     0.472879
Freq: D, dtype: float64
```

```
In [126]: ts_eastern.tz_convert('Europe/Berlin')
Out[126]:
2012-03-09 15:30:00+01:00    -0.202469
2012-03-10 15:30:00+01:00     0.050718
2012-03-11 14:30:00+01:00     0.639869
2012-03-12 14:30:00+01:00     0.597594
2012-03-13 14:30:00+01:00    -0.797246
2012-03-14 14:30:00+01:00     0.472879
Freq: D, dtype: float64
```

`tz_localize` and `tz_convert` are also instance methods on `DatetimeIndex`:

```
In [127]: ts.index.tz_localize('Asia/Shanghai')
Out[127]:
DatetimeIndex(['2012-03-09 09:30:00+08:00', '2012-03-10 09:30:00+08:00',
               '2012-03-11 09:30:00+08:00', '2012-03-12 09:30:00+08:00',
               '2012-03-13 09:30:00+08:00', '2012-03-14 09:30:00+08:00'],
              dtype='datetime64[ns, Asia/Shanghai]', freq='D')
```



Localizing naive timestamps also checks for ambiguous or non-existent times around daylight saving time transitions.



Time Zone 처리 > Operations with Time Zone–Aware Timestamp Objects

Time Zone Handling

Operations with Time Zone–Aware Timestamp Objects

Similar to time series and date ranges, individual `Timestamp` objects similarly can be localized from naive to time zone–aware and converted from one time zone to another:

```
In [128]: stamp = pd.Timestamp('2011-03-12 04:00')

In [129]: stamp_utc = stamp.tz_localize('utc')

In [130]: stamp_utc.tz_convert('America/New_York')
Out[130]: Timestamp('2011-03-11 23:00:00-0500', tz='America/New_York')
```

You can also pass a time zone when creating the `Timestamp`:

```
In [131]: stamp_moscow = pd.Timestamp('2011-03-12 04:00', tz='Europe/Moscow')

In [132]: stamp_moscow
Out[132]: Timestamp('2011-03-12 04:00:00+0300', tz='Europe/Moscow')
```

Time zone–aware `Timestamp` objects internally store a UTC timestamp value as nano-seconds since the Unix epoch (January 1, 1970); this UTC value is invariant between time zone conversions:

```
In [133]: stamp_utc.value
Out[133]: 1299902400000000000

In [134]: stamp_utc.tz_convert('America/New_York').value
Out[134]: 1299902400000000000
```

When performing time arithmetic using pandas's `DateOffset` objects, pandas respects daylight saving time transitions where possible. Here we construct timestamps that occur right before DST transitions (forward and backward). First, 30 minutes before transitioning to DST:

```
In [135]: from pandas.tseries.offsets import Hour

In [136]: stamp = pd.Timestamp('2012-03-12 01:30', tz='US/Eastern')

In [137]: stamp
Out[137]: Timestamp('2012-03-12 01:30:00-0400', tz='US/Eastern')

In [138]: stamp + Hour()
Out[138]: Timestamp('2012-03-12 02:30:00-0400', tz='US/Eastern')
```

Then, 90 minutes before transitioning out of DST:

```
In [139]: stamp = pd.Timestamp('2012-11-04 00:30', tz='US/Eastern')

In [140]: stamp
Out[140]: Timestamp('2012-11-04 00:30:00-0400', tz='US/Eastern')

In [141]: stamp + 2 * Hour()
Out[141]: Timestamp('2012-11-04 01:30:00-0500', tz='US/Eastern')
```



Time Zone 처리 > Operations Between Different Time Zones

Time Zone Handling

Operations Between Different Time Zones

If two time series with different time zones are combined, the result will be UTC. Since the timestamps are stored under the hood in UTC, this is a straightforward operation and requires no conversion to happen:

```
In [142]: rng = pd.date_range('3/7/2012 9:30', periods=10, freq='B')
```

```
In [143]: ts = pd.Series(np.random.randn(len(rng)), index=rng)
```

```
In [144]: ts
```

```
Out[144]:
```

```
2012-03-07 09:30:00    0.522356
2012-03-08 09:30:00   -0.546348
2012-03-09 09:30:00   -0.733537
2012-03-12 09:30:00    1.302736
2012-03-13 09:30:00    0.022199
2012-03-14 09:30:00    0.364287
2012-03-15 09:30:00   -0.922839
2012-03-16 09:30:00    0.312656
2012-03-19 09:30:00   -1.128497
2012-03-20 09:30:00   -0.333488
```

```
Freq: B, dtype: float64
```

```
In [145]: ts1 = ts[:7].tz_localize('Europe/London')
```

```
In [146]: ts2 = ts1[2:].tz_convert('Europe/Moscow')
```

```
In [147]: result = ts1 + ts2
```

```
In [148]: result.index
```

```
Out[148]:
```

```
DatetimeIndex(['2012-03-07 09:30:00+00:00', '2012-03-08 09:30:00+00:00',
               '2012-03-09 09:30:00+00:00', '2012-03-12 09:30:00+00:00',
               '2012-03-13 09:30:00+00:00', '2012-03-14 09:30:00+00:00',
               '2012-03-15 09:30:00+00:00'],
              dtype='datetime64[ns, UTC]', freq='B')
```

Q&A and Break Time

질의응답 및 휴식시간 (10분)



2부: 시계열

- 1) 기간 & 기간 연산
- 2) 리샘플링 & 빈도 변환
- 3) 윈도우 이동 함수





11.5 Periods and Period Arithmetic

Periods represent timespans, like days, months, quarters, or years. The `Period` class represents this data type, requiring a string or integer and a frequency from Table 11-4:

```
In [149]: p = pd.Period(2007, freq='A-DEC')
```

```
In [150]: p
Out[150]: Period('2007', 'A-DEC')
```

In this case, the `Period` object represents the full timespan from January 1, 2007, to December 31, 2007, inclusive. Conveniently, adding and subtracting integers from periods has the effect of shifting by their frequency:

```
In [151]: p + 5
Out[151]: Period('2012', 'A-DEC')
```

```
In [152]: p - 2
Out[152]: Period('2005', 'A-DEC')
```

If two periods have the same frequency, their difference is the number of units between them:

```
In [153]: pd.Period('2014', freq='A-DEC') - p
Out[153]: 7
```

Regular ranges of periods can be constructed with the `period_range` function:

```
In [154]: rng = pd.period_range('2000-01-01', '2000-06-30', freq='M')
```

```
In [155]: rng
Out[155]: PeriodIndex(['2000-01', '2000-02', '2000-03', '2000-04', '2000-05', '2000-06'], dtype='period[M]', freq='M')
```

The `PeriodIndex` class stores a sequence of periods and can serve as an axis index in any pandas data structure:

```
In [156]: pd.Series(np.random.randn(6), index=rng)
Out[156]:
2000-01    -0.514551
2000-02    -0.559782
2000-03    -0.783408
2000-04    -1.797685
2000-05    -0.172670
2000-06     0.680215
Freq: M, dtype: float64
```

If you have an array of strings, you can also use the `PeriodIndex` class:

```
In [157]: values = ['2001Q3', '2002Q2', '2003Q1']

In [158]: index = pd.PeriodIndex(values, freq='Q-DEC')

In [159]: index
Out[159]: PeriodIndex(['2001Q3', '2002Q2', '2003Q1'], dtype='period[Q-DEC]', freq='Q-DEC')
```


Period Frequency Conversion

Periods and PeriodIndex objects can be converted to another frequency with their `asfreq` method. As an example, suppose we had an annual period and wanted to convert it into a monthly period either at the start or end of the year. This is fairly straightforward:

```
In [160]: p = pd.Period('2007', freq='A-DEC')
```

```
In [161]: p  
Out[161]: Period('2007', 'A-DEC')
```

```
In [162]: p.asfreq('M', how='start')  
Out[162]: Period('2007-01', 'M')
```

```
In [163]: p.asfreq('M', how='end')  
Out[163]: Period('2007-12', 'M')
```

You can think of `Period('2007', 'A-DEC')` as being a sort of cursor pointing to a span of time, subdivided by monthly periods. See [Figure 11-1](#) for an illustration of this. For a *fiscal year* ending on a month other than December, the corresponding monthly subperiods are different:

```
In [164]: p = pd.Period('2007', freq='A-JUN')
```

```
In [165]: p  
Out[165]: Period('2007', 'A-JUN')
```

```
In [166]: p.asfreq('M', 'start')  
Out[166]: Period('2006-07', 'M')
```

```
In [167]: p.asfreq('M', 'end')  
Out[167]: Period('2007-06', 'M')
```

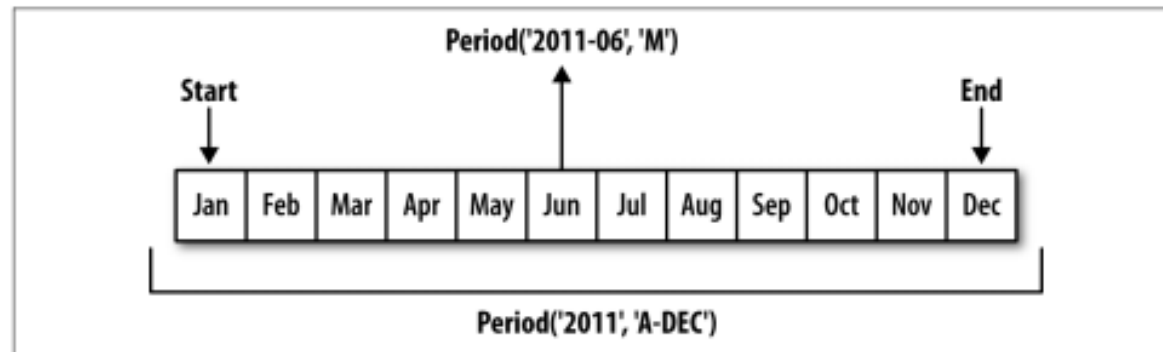


Figure 11-1. Period frequency conversion illustration

기간 & 기간 연산 > Period Frequency Conversion

Periods and Period Arithmetic

When you are converting from high to low frequency, pandas determines the super-period depending on where the subperiod “belongs.” For example, in A-JUN frequency, the month Aug-2007 is actually part of the 2008 period:

```
In [168]: p = pd.Period('Aug-2007', 'M')
```

```
In [169]: p.asfreq('A-JUN')
```

```
Out[169]: Period('2008', 'A-JUN')
```

Whole PeriodIndex objects or time series can be similarly converted with the same semantics:

```
In [170]: rng = pd.period_range('2006', '2009', freq='A-DEC')
```

```
In [171]: ts = pd.Series(np.random.randn(len(rng)), index=rng)
```

```
In [172]: ts
```

```
Out[172]:
```

```
2006    1.607578
```

```
2007    0.200381
```

```
2008   -0.834068
```

```
2009   -0.302988
```

```
Freq: A-DEC, dtype: float64
```

```
In [173]: ts.asfreq('M', how='start')
```

```
Out[173]:
```

```
2006-01    1.607578
```

```
2007-01    0.200381
```

```
2008-01   -0.834068
```

```
2009-01   -0.302988
```

```
Freq: M, dtype: float64
```

Here, the annual periods are replaced with monthly periods corresponding to the first month falling within each annual period. If we instead wanted the last business day of each year, we can use the 'B' frequency and indicate that we want the end of the period:

```
In [174]: ts.asfreq('B', how='end')
```

```
Out[174]:
```

```
2006-12-29    1.607578
```

```
2007-12-31    0.200381
```

```
2008-12-31   -0.834068
```

```
2009-12-31   -0.302988
```

```
Freq: B, dtype: float64
```

Quarterly Period Frequencies

Quarterly data is standard in accounting, finance, and other fields. Much quarterly data is reported relative to a *fiscal year end*, typically the last calendar or business day of one of the 12 months of the year. Thus, the period 2012Q4 has a different meaning depending on fiscal year end. pandas supports all 12 possible quarterly frequencies as Q-JAN through Q-DEC:

```
In [175]: p = pd.Period('2012Q4', freq='Q-JAN')
```

```
In [176]: p
Out[176]: Period('2012Q4', 'Q-JAN')
```

In the case of fiscal year ending in January, 2012Q4 runs from November through January, which you can check by converting to daily frequency. See [Figure 11-2](#) for an illustration.

Year 2012												
M	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC
Q-DEC	2012Q1			2012Q2			2012Q3			2012Q4		
Q-SEP	2012Q2			2012Q3			2012Q4			2013Q1		
Q-FEB	2012Q4			2013Q1			2013Q2			2013Q3		
										Q4		

Figure 11-2. Different quarterly frequency conventions

```
In [177]: p.asfreq('D', 'start')
Out[177]: Period('2011-11-01', 'D')
```

```
In [178]: p.asfreq('D', 'end')
Out[178]: Period('2012-01-31', 'D')
```

Thus, it's possible to do easy period arithmetic; for example, to get the timestamp at 4 PM on the second-to-last business day of the quarter, you could do:

```
In [179]: p4pm = (p.asfreq('B', 'e') - 1).asfreq('T', 's') + 16 * 60
```

```
In [180]: p4pm
Out[180]: Period('2012-01-30 16:00', 'T')
```

```
In [181]: p4pm.to_timestamp()
Out[181]: Timestamp('2012-01-30 16:00:00')
```

기간 & 기간 연산 > Quarterly Period Frequencies

Periods and Period Arithmetic

You can generate quarterly ranges using `period_range`. Arithmetic is identical, too:

```
In [182]: rng = pd.period_range('2011Q3', '2012Q4', freq='Q-JAN')

In [183]: ts = pd.Series(np.arange(len(rng)), index=rng)

In [184]: ts
Out[184]:
2011Q3      0
2011Q4      1
2012Q1      2
2012Q2      3
2012Q3      4
2012Q4      5
Freq: Q-JAN, dtype: int64

In [185]: new_rng = (rng.asfreq('B', 'e') - 1).asfreq('T', 's') + 16 * 60

In [186]: ts.index = new_rng.to_timestamp()

In [187]: ts
Out[187]:
2010-10-28 16:00:00      0
2011-01-28 16:00:00      1
2011-04-28 16:00:00      2
2011-07-28 16:00:00      3
2011-10-28 16:00:00      4
2012-01-30 16:00:00      5
dtype: int64
```



기간 & 기간 연산 > Converting Timestamps to Periods (and Back)

Converting Timestamps to Periods (and Back)

Series and DataFrame objects indexed by timestamps can be converted to periods with the `to_period` method:

```
In [188]: rng = pd.date_range('2000-01-01', periods=3, freq='M')
```

```
In [189]: ts = pd.Series(np.random.randn(3), index=rng)
```

```
In [190]: ts
```

```
Out[190]:
```

```
2000-01-31    1.663261
```

```
2000-02-29   -0.996206
```

```
2000-03-31    1.521760
```

```
Freq: M, dtype: float64
```

```
In [191]: pts = ts.to_period()
```

```
In [192]: pts
```

```
Out[192]:
```

```
2000-01    1.663261
```

```
2000-02   -0.996206
```

```
2000-03    1.521760
```

```
Freq: M, dtype: float64
```

Since periods refer to non-overlapping timespans, a timestamp can only belong to a single period for a given frequency. While the frequency of the new `PeriodIndex` is inferred from the timestamps by default, you can specify any frequency you want. There is also no problem with having duplicate periods in the result:

```
In [193]: rng = pd.date_range('1/29/2000', periods=6, freq='D')
```

```
In [194]: ts2 = pd.Series(np.random.randn(6), index=rng)
```

```
In [195]: ts2
```

```
Out[195]:
```

```
2000-01-29    0.244175
```

```
2000-01-30    0.423331
```

```
2000-01-31   -0.654040
```

```
2000-02-01    2.089154
```

```
2000-02-02   -0.060220
```

```
2000-02-03   -0.167933
```

```
Freq: D, dtype: float64
```

```
In [196]: ts2.to_period('M')
```

```
Out[196]:
```

```
2000-01    0.244175
```

```
2000-01    0.423331
```

```
2000-01   -0.654040
```

```
2000-02    2.089154
```

```
2000-02   -0.060220
```

```
2000-02   -0.167933
```

```
Freq: M, dtype: float64
```

To convert back to timestamps, use `to_timestamp`:

```
In [197]: pts = ts2.to_period()
```

```
In [198]: pts
```

```
Out[198]:
```

```
2000-01-29    0.244175
```

```
2000-01-30    0.423331
```

```
2000-01-31   -0.654040
```

```
2000-02-01    2.089154
```

```
2000-02-02   -0.060220
```

```
2000-02-03   -0.167933
```

```
Freq: D, dtype: float64
```

```
In [199]: pts.to_timestamp(how='end')
```

```
Out[199]:
```

```
2000-01-29    0.244175
```

```
2000-01-30    0.423331
```

```
2000-01-31   -0.654040
```

```
2000-02-01    2.089154
```

```
2000-02-02   -0.060220
```

```
2000-02-03   -0.167933
```

```
Freq: D, dtype: float64
```




기간 & 기간 연산 > Creating a PeriodIndex from Arrays

Periods and Period Arithmetic

Creating a PeriodIndex from Arrays

Fixed frequency datasets are sometimes stored with timespan information spread across multiple columns. For example, in this macroeconomic dataset, the year and quarter are in different columns:

```
In [200]: data = pd.read_csv('examples/macrodta.csv')
```

```
In [201]: data.head(5)
```

```
Out[201]:
```

	year	quarter	realgdp	realcons	realinv	realgovt	realdpi	cpi	\
0	1959.0	1.0	2710.349	1707.4	286.898	470.045	1886.9	28.98	
1	1959.0	2.0	2778.801	1733.7	310.859	481.301	1919.7	29.15	
2	1959.0	3.0	2775.488	1751.8	289.226	491.260	1916.4	29.35	
3	1959.0	4.0	2785.204	1753.7	299.356	484.052	1931.3	29.37	
4	1960.0	1.0	2847.699	1770.5	331.722	462.199	1955.5	29.54	

	m1	tbilrate	unemp	pop	infl	realint
0	139.7	2.82	5.8	177.146	0.00	0.00
1	141.7	3.08	5.1	177.830	2.34	0.74
2	140.5	3.82	5.3	178.657	2.74	1.09
3	140.0	4.33	5.6	179.386	0.27	4.06
4	139.6	3.50	5.2	180.007	2.31	1.19

```
In [202]: data.year
```

```
Out[202]:
```

```
0    1959.0
1    1959.0
2    1959.0
3    1959.0
4    1960.0
5    1960.0
6    1960.0
7    1960.0
8    1961.0
9    1961.0
```

```
...
```

```
193    2007.0
194    2007.0
195    2007.0
196    2008.0
197    2008.0
198    2008.0
199    2008.0
200    2009.0
201    2009.0
202    2009.0
```

```
Name: year, Length: 203, dtype:
```

```
In [203]: data.quarter
```

```
Out[203]:
```

```
0    1.0
1    2.0
2    3.0
3    4.0
4    1.0
5    2.0
6    3.0
7    4.0
8    1.0
9    2.0
```

```
...
```

```
193    2.0
194    3.0
195    4.0
196    1.0
197    2.0
198    3.0
199    4.0
200    1.0
201    2.0
202    3.0
```

```
Name: quarter, Length: 203, dtype: float64
```



기간 & 기간 연산 > *Creating a PeriodIndex from Arrays*

Periods and Period Arithmetic

By passing these arrays to `PeriodIndex` with a frequency, you can combine them to form an index for the DataFrame:

```
In [204]: index = pd.PeriodIndex(year=data.year, quarter=data.quarter,  
.....:                          freq='Q-DEC')
```

```
In [205]: index
```

```
Out[205]:
```

```
PeriodIndex(['1959Q1', '1959Q2', '1959Q3', '1959Q4', '1960Q1', '1960Q2',  
            '1960Q3', '1960Q4', '1961Q1', '1961Q2',  
            ...,  
            '2007Q2', '2007Q3', '2007Q4', '2008Q1', '2008Q2', '2008Q3',  
            '2008Q4', '2009Q1', '2009Q2', '2009Q3'],  
            dtype='period[Q-DEC]', length=203, freq='Q-DEC')
```

```
In [206]: data.index = index
```

```
In [207]: data.infl
```

```
Out[207]:
```

```
1959Q1    0.00  
1959Q2    2.34  
1959Q3    2.74  
1959Q4    0.27  
1960Q1    2.31  
1960Q2    0.14  
1960Q3    2.70  
1960Q4    1.21  
1961Q1   -0.40  
1961Q2    1.47
```

```
...  
2007Q2    2.75  
2007Q3    3.45  
2007Q4    6.38  
2008Q1    2.82  
2008Q2    8.53  
2008Q3   -3.16  
2008Q4   -8.79  
2009Q1    0.94  
2009Q2    3.37  
2009Q3    3.56
```

```
Freq: Q-DEC, Name: infl, Length: 203, dtype: float64
```



11.6 Resampling and Frequency Conversion

Resampling refers to the process of converting a time series from one frequency to another. Aggregating higher frequency data to lower frequency is called *downsampling*, while converting lower frequency to higher frequency is called *upsampling*. Not all resampling falls into either of these categories; for example, converting W-WED (weekly on Wednesday) to W-FRI is neither upsampling nor downsampling.

pandas objects are equipped with a `resample` method, which is the workhorse function for all frequency conversion. `resample` has a similar API to `groupby`; you call `resample` to group the data, then call an aggregation function:

```
In [208]: rng = pd.date_range('2000-01-01', periods=100, freq='D')
```

```
In [209]: ts = pd.Series(np.random.randn(len(rng)), index=rng)
```

```
In [210]: ts
```

```
Out[210]:
```

```
2000-01-01    0.631634
2000-01-02   -1.594313
2000-01-03   -1.519937
2000-01-04    1.108752
2000-01-05    1.255853
2000-01-06   -0.024330
2000-01-07   -2.047939
2000-01-08   -0.272657
2000-01-09   -1.692615
2000-01-10    1.423830
```

```
...
```

```
2000-03-31   -0.007852
2000-04-01   -1.638806
2000-04-02    1.401227
2000-04-03    1.758539
2000-04-04    0.628932
2000-04-05   -0.423776
2000-04-06    0.789740
2000-04-07    0.937568
2000-04-08   -2.253294
2000-04-09   -1.772919
```

```
Freq: D, Length: 100, dtype: float64
```

```
In [211]: ts.resample('M').mean()
```

```
Out[211]:
```

```
2000-01-31   -0.165893
2000-02-29    0.078606
2000-03-31    0.223811
2000-04-30   -0.063643
```

```
Freq: M, dtype: float64
```

```
In [212]: ts.resample('M', kind='period').mean()
```

```
Out[212]:
```

```
2000-01   -0.165893
2000-02    0.078606
2000-03    0.223811
2000-04   -0.063643
```

```
Freq: M, dtype: float64
```

`resample` is a flexible and high-performance method that can be used to process very large time series. The examples in the following sections illustrate its semantics and use. Table 11-5 summarizes some of its options.

Table 11-5. *Resample method arguments*

Argument	Description
<code>freq</code>	String or DateOffset indicating desired resampled frequency (e.g., 'M', '5min', or <code>Second(15)</code>)
<code>axis</code>	Axis to resample on; default <code>axis=0</code>
<code>fill_method</code>	How to interpolate when upsampling, as in 'ffill' or 'bfill'; by default does no interpolation
<code>closed</code>	In downsampling, which end of each interval is closed (inclusive), 'right' or 'left'
<code>label</code>	In downsampling, how to label the aggregated result, with the 'right' or 'left' bin edge (e.g., the 9:30 to 9:35 five-minute interval could be labeled 9:30 or 9:35)
<code>loffset</code>	Time adjustment to the bin labels, such as '-1s' / <code>Second(-1)</code> to shift the aggregate labels one second earlier
<code>limit</code>	When forward or backward filling, the maximum number of periods to fill
<code>kind</code>	Aggregate to periods ('period') or timestamps ('timestamp'); defaults to the type of index the time series has
<code>convention</code>	When resampling periods, the convention ('start' or 'end') for converting the low-frequency period to high frequency; defaults to 'end'



리샘플링 & 빈도 변환 > Downsampling

Resampling and Frequency Conversion

Downsampling

Aggregating data to a regular, lower frequency is a pretty normal time series task. The data you're aggregating doesn't need to be fixed frequently; the desired frequency defines *bin edges* that are used to slice the time series into pieces to aggregate. For example, to convert to monthly, 'M' or 'BM', you need to chop up the data into one-month intervals. Each interval is said to be *half-open*; a data point can only belong to one interval, and the union of the intervals must make up the whole time frame. There are a couple things to think about when using `resample` to downsample data:

- Which side of each interval is *closed*
- How to label each aggregated bin, either with the start of the interval or the end

To illustrate, let's look at some one-minute data:

```
In [213]: rng = pd.date_range('2000-01-01', periods=12, freq='T')
```

```
In [214]: ts = pd.Series(np.arange(12), index=rng)
```

```
In [215]: ts
```

```
Out[215]:
```

```

2000-01-01 00:00:00    0
2000-01-01 00:01:00    1
2000-01-01 00:02:00    2
2000-01-01 00:03:00    3
2000-01-01 00:04:00    4
2000-01-01 00:05:00    5
2000-01-01 00:06:00    6
2000-01-01 00:07:00    7
2000-01-01 00:08:00    8
2000-01-01 00:09:00    9
2000-01-01 00:10:00   10
2000-01-01 00:11:00   11
Freq: T, dtype: int64

```

Suppose you wanted to aggregate this data into five-minute chunks or *bars* by taking the sum of each group:

```

In [216]: ts.resample('5min', closed='right').sum()
Out[216]:
1999-12-31 23:55:00    0
2000-01-01 00:00:00   15
2000-01-01 00:05:00   40
2000-01-01 00:10:00   11
Freq: 5T, dtype: int64

```

The frequency you pass defines bin edges in five-minute increments. By default, the *left* bin edge is inclusive, so the 00:00 value is included in the 00:00 to 00:05 interval.¹ Passing `closed='right'` changes the interval to be closed on the right:

```

In [217]: ts.resample('5min', closed='right').sum()
Out[217]:
1999-12-31 23:55:00    0
2000-01-01 00:00:00   15
2000-01-01 00:05:00   40
2000-01-01 00:10:00   11
Freq: 5T, dtype: int64

```

The resulting time series is labeled by the timestamps from the left side of each bin. By passing `label='right'` you can label them with the right bin edge:

```

In [218]: ts.resample('5min', closed='right', label='right').sum()
Out[218]:
2000-01-01 00:00:00    0
2000-01-01 00:05:00   15
2000-01-01 00:10:00   40
2000-01-01 00:15:00   11
Freq: 5T, dtype: int64

```

¹ The choice of the default values for `closed` and `label` might seem a bit odd to some users. In practice the choice is somewhat arbitrary; for some target frequencies, `closed='left'` is preferable, while for others `closed='right'` makes more sense. The important thing is that you keep in mind exactly how you are segmenting the data.



리샘플링 & 빈도 변환 > Downsampling

Resampling and Frequency Conversion

See **Figure 11-3** for an illustration of minute frequency data being resampled to five-minute frequency.

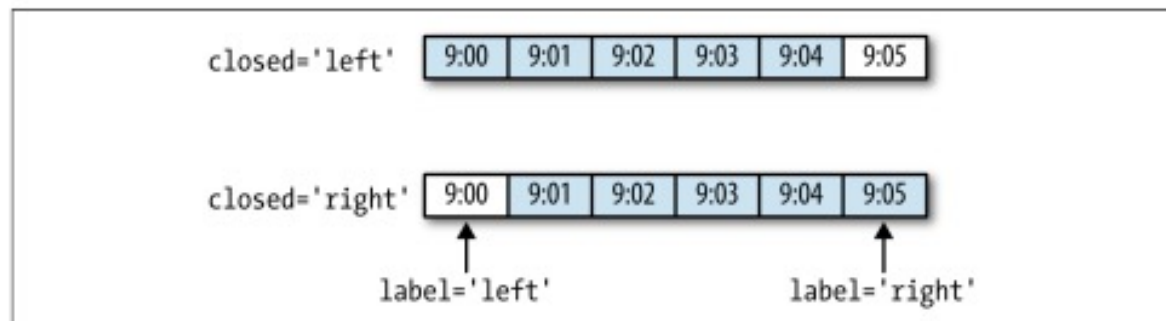


Figure 11-3. Five-minute resampling illustration of closed, label conventions

Lastly, you might want to shift the result index by some amount, say subtracting one second from the right edge to make it more clear which interval the timestamp refers to. To do this, pass a string or date offset to `loffset`:

```
In [219]: ts.resample('5min', closed='right',
.....:               label='right', loffset='-1s').sum()
Out[219]:
1999-12-31 23:59:59    0
2000-01-01 00:04:59    15
2000-01-01 00:09:59    40
2000-01-01 00:14:59    11
Freq: 5T, dtype: int64
```

You also could have accomplished the effect of `loffset` by calling the `shift` method on the result without the `loffset`.

Open-High-Low-Close (OHLC) resampling

In finance, a popular way to aggregate a time series is to compute four values for each bucket: the first (open), last (close), maximum (high), and minimal (low) values. By using the `ohlc` aggregate function you will obtain a DataFrame having columns containing these four aggregates, which are efficiently computed in a single sweep of the data:

```
In [220]: ts.resample('5min').ohlc()
Out[220]:
```

	open	high	low	close
2000-01-01 00:00:00	0	4	0	4
2000-01-01 00:05:00	5	9	5	9
2000-01-01 00:10:00	10	11	10	11



리샘플링 & 빈도 변환 > Upsampling and Interpolation

Resampling and Frequency Conversion

Upsampling and Interpolation

When converting from a low frequency to a higher frequency, no aggregation is needed. Let's consider a DataFrame with some weekly data:

```
In [221]: frame = pd.DataFrame(np.random.randn(2, 4),
.....:                        index=pd.date_range('1/1/2000', periods=2,
.....:                        freq='W-WED'),
.....:                        columns=['Colorado', 'Texas', 'New York', 'Ohio'])
```

```
In [222]: frame
```

```
Out[222]:
```

	Colorado	Texas	New York	Ohio
2000-01-05	-0.896431	0.677263	0.036503	0.087102
2000-01-12	-0.046662	0.927238	0.482284	-0.867130

When you are using an aggregation function with this data, there is only one value per group, and missing values result in the gaps. We use the `asfreq` method to convert to the higher frequency without any aggregation:

```
In [223]: df_daily = frame.resample('D').asfreq()
```

```
In [224]: df_daily
```

```
Out[224]:
```

	Colorado	Texas	New York	Ohio
2000-01-05	-0.896431	0.677263	0.036503	0.087102
2000-01-06	NaN	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN	NaN
2000-01-08	NaN	NaN	NaN	NaN
2000-01-09	NaN	NaN	NaN	NaN
2000-01-10	NaN	NaN	NaN	NaN
2000-01-11	NaN	NaN	NaN	NaN
2000-01-12	-0.046662	0.927238	0.482284	-0.867130

Suppose you wanted to fill forward each weekly value on the non-Wednesdays. The same filling or interpolation methods available in the `fillna` and `reindex` methods are available for resampling:

```
In [225]: frame.resample('D').ffill()
```

```
Out[225]:
```

	Colorado	Texas	New York	Ohio
2000-01-05	-0.896431	0.677263	0.036503	0.087102
2000-01-06	-0.896431	0.677263	0.036503	0.087102
2000-01-07	-0.896431	0.677263	0.036503	0.087102
2000-01-08	-0.896431	0.677263	0.036503	0.087102
2000-01-09	-0.896431	0.677263	0.036503	0.087102
2000-01-10	-0.896431	0.677263	0.036503	0.087102
2000-01-11	-0.896431	0.677263	0.036503	0.087102
2000-01-12	-0.046662	0.927238	0.482284	-0.867130

You can similarly choose to only fill a certain number of periods forward to limit how far to continue using an observed value:

```
In [226]: frame.resample('D').ffill(limit=2)
```

```
Out[226]:
```

	Colorado	Texas	New York	Ohio
2000-01-05	-0.896431	0.677263	0.036503	0.087102
2000-01-06	-0.896431	0.677263	0.036503	0.087102
2000-01-07	-0.896431	0.677263	0.036503	0.087102
2000-01-08	NaN	NaN	NaN	NaN
2000-01-09	NaN	NaN	NaN	NaN
2000-01-10	NaN	NaN	NaN	NaN
2000-01-11	NaN	NaN	NaN	NaN
2000-01-12	-0.046662	0.927238	0.482284	-0.867130

Notably, the new date index need not overlap with the old one at all:

```
In [227]: frame.resample('W-THU').ffill()
```

```
Out[227]:
```

	Colorado	Texas	New York	Ohio
2000-01-06	-0.896431	0.677263	0.036503	0.087102
2000-01-13	-0.046662	0.927238	0.482284	-0.867130

리샘플링 & 빈도 변환 > Resampling with Periods

Resampling and Frequency Conversion

Resampling with Periods

Resampling data indexed by periods is similar to timestamps:

```
In [228]: frame = pd.DataFrame(np.random.randn(24, 4),
.....:                        index=pd.period_range('1-2000', '12-2001',
.....:                        freq='M'),
.....:                        columns=['Colorado', 'Texas', 'New York', 'Ohio'])
```

```
In [229]: frame[:5]
```

```
Out[229]:
```

	Colorado	Texas	New York	Ohio
2000-01	0.493841	-0.155434	1.397286	1.507055
2000-02	-1.179442	0.443171	1.395676	-0.529658
2000-03	0.787358	0.248845	0.743239	1.267746
2000-04	1.302395	-0.272154	-0.051532	-0.467740
2000-05	-1.040816	0.426419	0.312945	-1.115689

```
In [230]: annual_frame = frame.resample('A-DEC').mean()
```

```
In [231]: annual_frame
```

```
Out[231]:
```

	Colorado	Texas	New York	Ohio
2000	0.556703	0.016631	0.111873	-0.027445
2001	0.046303	0.163344	0.251503	-0.157276

Upsampling is more nuanced, as you must make a decision about which end of the timespan in the new frequency to place the values before resampling, just like the `asfreq` method. The `convention` argument defaults to 'start' but can also be 'end':

```
# Q-DEC: Quarterly, year ending in December
In [232]: annual_frame.resample('Q-DEC').ffill()
Out[232]:
```

	Colorado	Texas	New York	Ohio
2000Q1	0.556703	0.016631	0.111873	-0.027445
2000Q2	0.556703	0.016631	0.111873	-0.027445
2000Q3	0.556703	0.016631	0.111873	-0.027445
2000Q4	0.556703	0.016631	0.111873	-0.027445
2001Q1	0.046303	0.163344	0.251503	-0.157276
2001Q2	0.046303	0.163344	0.251503	-0.157276
2001Q3	0.046303	0.163344	0.251503	-0.157276
2001Q4	0.046303	0.163344	0.251503	-0.157276

```
In [233]: annual_frame.resample('Q-DEC', convention='end').ffill()
```

```
Out[233]:
```

	Colorado	Texas	New York	Ohio
2000Q4	0.556703	0.016631	0.111873	-0.027445
2001Q1	0.556703	0.016631	0.111873	-0.027445
2001Q2	0.556703	0.016631	0.111873	-0.027445
2001Q3	0.556703	0.016631	0.111873	-0.027445
2001Q4	0.046303	0.163344	0.251503	-0.157276

Since periods refer to timespans, the rules about upsampling and downsampling are more rigid:

- In downsampling, the target frequency must be a *subperiod* of the source frequency.
- In upsampling, the target frequency must be a *superperiod* of the source frequency.

If these rules are not satisfied, an exception will be raised. This mainly affects the quarterly, annual, and weekly frequencies; for example, the timespans defined by Q-MAR only line up with A-MAR, A-JUN, A-SEP, and A-DEC:

```
In [234]: annual_frame.resample('Q-MAR').ffill()
```

```
Out[234]:
```

	Colorado	Texas	New York	Ohio
2000Q4	0.556703	0.016631	0.111873	-0.027445
2001Q1	0.556703	0.016631	0.111873	-0.027445
2001Q2	0.556703	0.016631	0.111873	-0.027445
2001Q3	0.556703	0.016631	0.111873	-0.027445
2001Q4	0.046303	0.163344	0.251503	-0.157276
2002Q1	0.046303	0.163344	0.251503	-0.157276
2002Q2	0.046303	0.163344	0.251503	-0.157276
2002Q3	0.046303	0.163344	0.251503	-0.157276



11.7 Moving Window Functions

An important class of array transformations used for time series operations are statistics and other functions evaluated over a sliding window or with exponentially decaying weights. This can be useful for smoothing noisy or gappy data. I call these *moving window functions*, even though it includes functions without a fixed-length window like exponentially weighted moving average. Like other statistical functions, these also automatically exclude missing data.

Before digging in, we can load up some time series data and resample it to business day frequency:

```
In [235]: close_px_all = pd.read_csv('examples/stock_px_2.csv',
.....:                               parse_dates=True, index_col=0)

In [236]: close_px = close_px_all[['AAPL', 'MSFT', 'XOM']]

In [237]: close_px = close_px.resample('B').ffill()
```

I now introduce the rolling operator, which behaves similarly to `resample` and `groupby`. It can be called on a Series or DataFrame along with a window (expressed as a number of periods; see [Figure 11-4](#) for the plot created):

```
In [238]: close_px.AAPL.plot()
Out[238]: <matplotlib.axes._subplots.AxesSubplot at 0x7f2f2570cf98>

In [239]: close_px.AAPL.rolling(250).mean().plot()
```

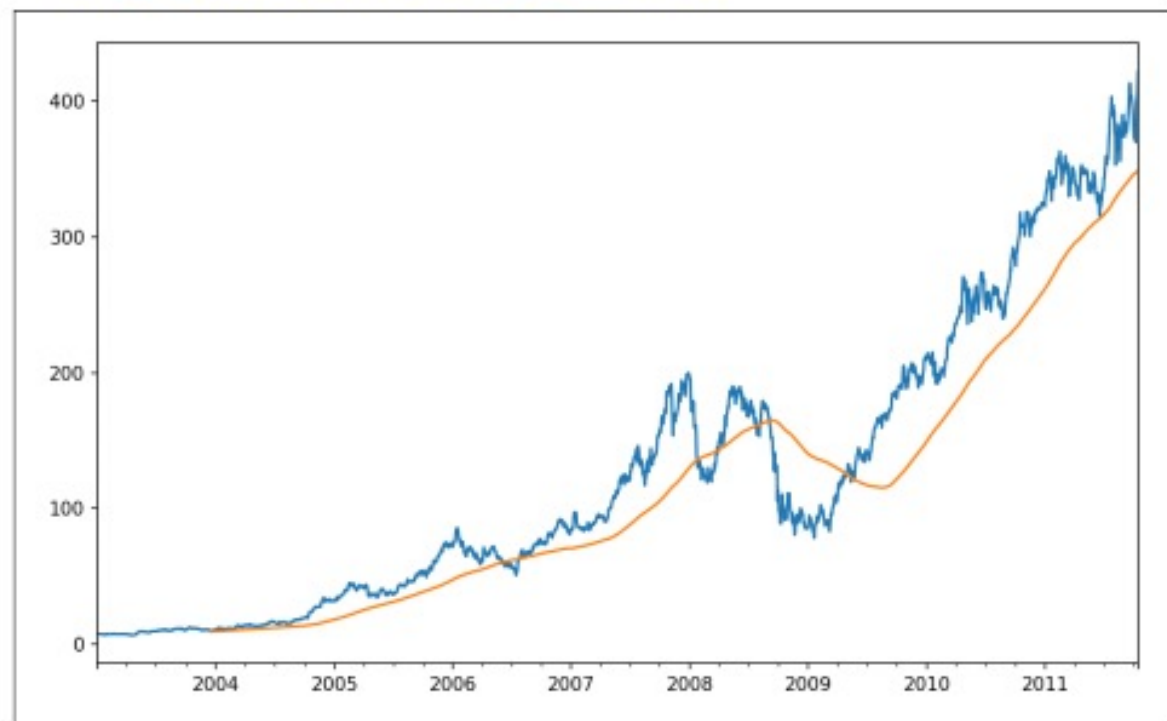


Figure 11-4. Apple Price with 250-day MA



윈도우 이동 함수

Moving Window Functions

The expression `rolling(250)` is similar in behavior to `groupby`, but instead of grouping it creates an object that enables grouping over a 250-day sliding window. So here we have the 250-day moving window average of Apple's stock price.

By default rolling functions require all of the values in the window to be non-NA. This behavior can be changed to account for missing data and, in particular, the fact that you will have fewer than window periods of data at the beginning of the time series (see [Figure 11-5](#)):

```
In [241]: appl_std250 = close_px.AAPL.rolling(250, min_periods=10).std()
```

```
In [242]: appl_std250[5:12]
```

```
Out[242]:
```

2003-01-09	NaN
2003-01-10	NaN
2003-01-13	NaN
2003-01-14	NaN
2003-01-15	0.077496
2003-01-16	0.074760
2003-01-17	0.112368

```
Freq: B, Name: AAPL, dtype: float64
```

```
In [243]: appl_std250.plot()
```

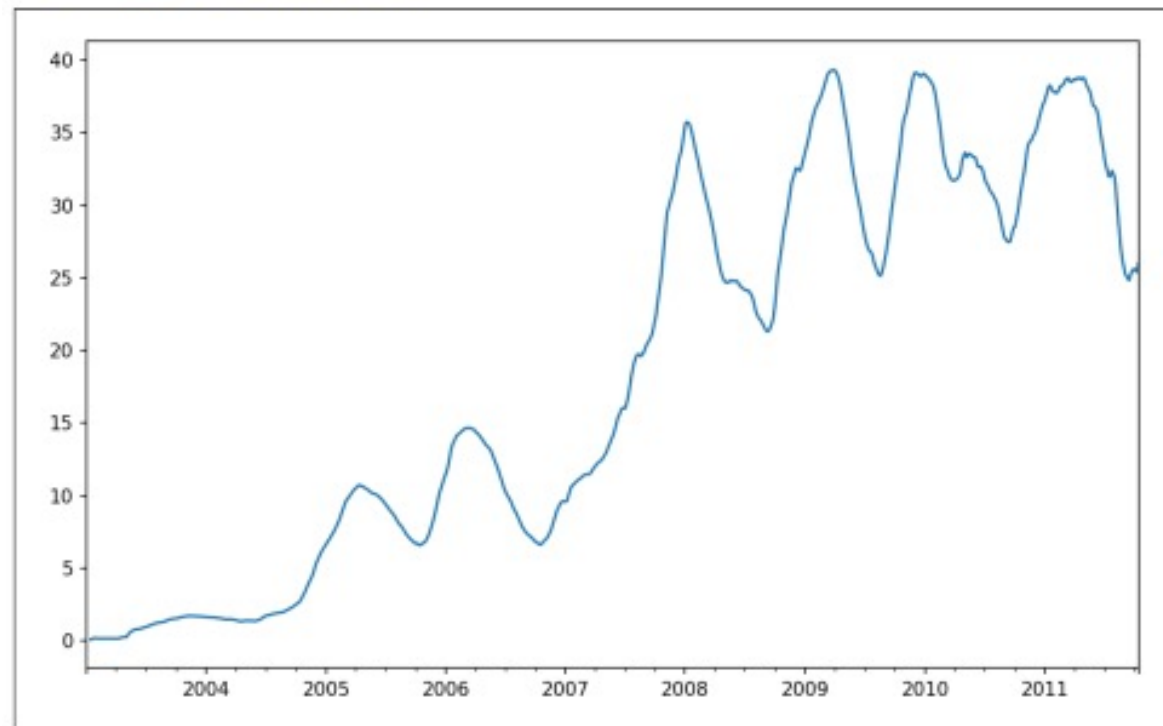


Figure 11-5. Apple 250-day daily return standard deviation



윈도우 이동 함수

Moving Window Functions

In order to compute an *expanding window mean*, use the expanding operator instead of rolling. The expanding mean starts the time window from the beginning of the time series and increases the size of the window until it encompasses the whole series. An expanding window mean on the `apple_std250` time series looks like this:

Calling a moving window function on a DataFrame applies the transformation to each column (see Figure 11-6):

```
In [246]: close_px.rolling(60).mean().plot(logy=True)
```

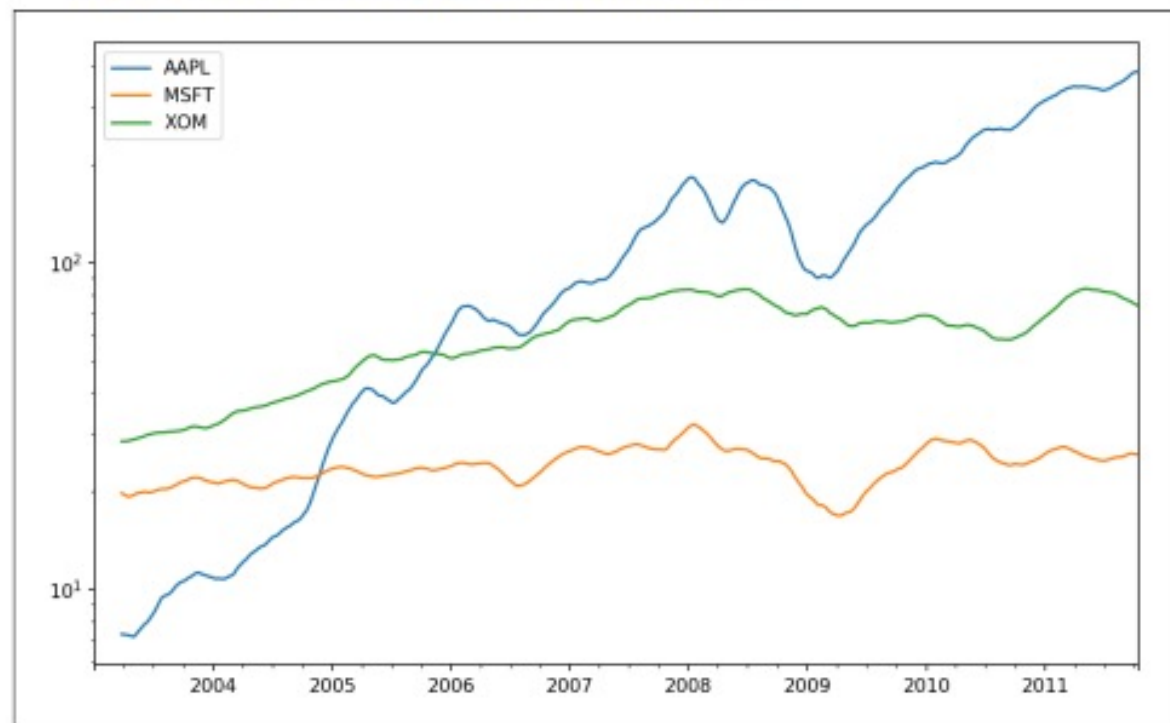


Figure 11-6. Stocks prices 60-day MA (log Y-axis)

The rolling function also accepts a string indicating a fixed-size time offset rather than a set number of periods. Using this notation can be useful for irregular time series. These are the same strings that you can pass to `resample`. For example, we could compute a 20-day rolling mean like so:

```
In [247]: close_px.rolling('20D').mean()
```

```
Out[247]:
```

	AAPL	MSFT	XOM
2003-01-02	7.400000	21.110000	29.220000
2003-01-03	7.425000	21.125000	29.230000
2003-01-06	7.433333	21.256667	29.473333
2003-01-07	7.432500	21.425000	29.342500
2003-01-08	7.402000	21.402000	29.240000
2003-01-09	7.391667	21.490000	29.273333
2003-01-10	7.387143	21.558571	29.238571
2003-01-13	7.378750	21.633750	29.197500
2003-01-14	7.370000	21.717778	29.194444
2003-01-15	7.355000	21.757000	29.152000
...	...	...	...
2011-10-03	398.002143	25.890714	72.413571
2011-10-04	396.802143	25.807857	72.427143
2011-10-05	395.751429	25.729286	72.422857
2011-10-06	394.099286	25.673571	72.375714
2011-10-07	392.479333	25.712000	72.454667
2011-10-10	389.351429	25.602143	72.527857
2011-10-11	388.505000	25.674286	72.835000
2011-10-12	388.531429	25.810000	73.400714
2011-10-13	388.826429	25.961429	73.905000
2011-10-14	391.038000	26.048667	74.185333

[2292 rows x 3 columns]



윈도우 이동 함수 > Exponentially Weighted Functions

Moving Window Functions

Exponentially Weighted Functions

An alternative to using a static window size with equally weighted observations is to specify a constant *decay factor* to give more weight to more recent observations. There are a couple of ways to specify the decay factor. A popular one is using a *span*, which makes the result comparable to a simple moving window function with window size equal to the span.

Since an exponentially weighted statistic places more weight on more recent observations, it “adapts” faster to changes compared with the equal-weighted version.

pandas has the `ewm` operator to go along with rolling and expanding. Here’s an example comparing a 60-day moving average of Apple’s stock price with an EW moving average with `span=60` (see Figure 11-7):

```
In [249]: aapl_px = close_px.AAPL['2006':'2007']

In [250]: ma60 = aapl_px.rolling(30, min_periods=20).mean()

In [251]: ewma60 = aapl_px.ewm(span=30).mean()

In [252]: ma60.plot(style='k--', label='Simple MA')
Out[252]: <matplotlib.axes._subplots.AxesSubplot at 0x7f2f252161d0>

In [253]: ewma60.plot(style='k-', label='EW MA')
Out[253]: <matplotlib.axes._subplots.AxesSubplot at 0x7f2f252161d0>

In [254]: plt.legend()
```

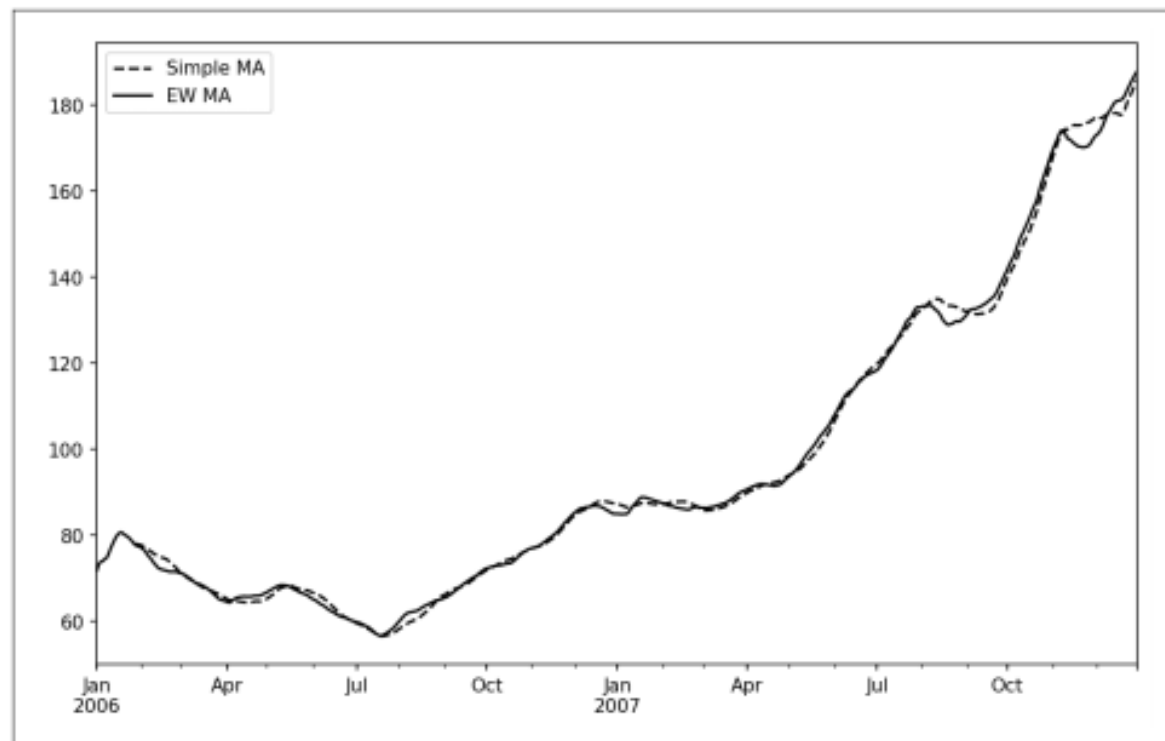


Figure 11-7. Simple moving average versus exponentially weighted



윈도우 이동 함수 > Binary Moving Window Functions

Moving Window Functions

Binary Moving Window Functions

Some statistical operators, like correlation and covariance, need to operate on two time series. As an example, financial analysts are often interested in a stock's correlation to a benchmark index like the S&P 500. To have a look at this, we first compute the percent change for all of our time series of interest:

```
In [256]: spx_px = close_px_all['SPX']
```

```
In [257]: spx_rets = spx_px.pct_change()
```

```
In [258]: returns = close_px.pct_change()
```

The `corr` aggregation function after we call `rolling` can then compute the rolling correlation with `spx_rets` (see Figure 11-8 for the resulting plot):

```
In [259]: corr = returns.AAPL.rolling(125, min_periods=100).corr(spx_rets)
```

```
In [260]: corr.plot()
```

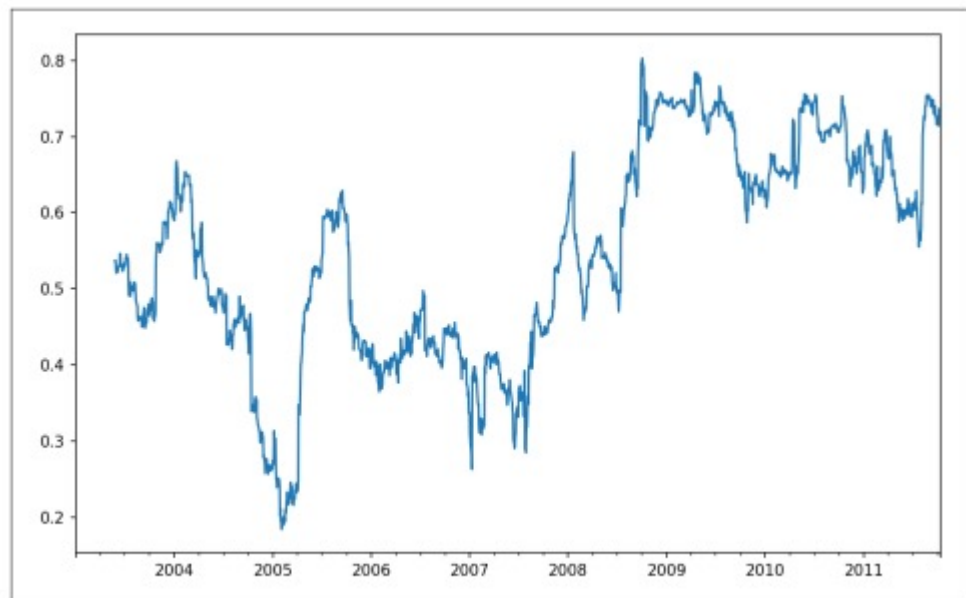


Figure 11-8. Six-month AAPL return correlation to S&P 500

Suppose you wanted to compute the correlation of the S&P 500 index with many stocks at once. Writing a loop and creating a new DataFrame would be easy but might get repetitive, so if you pass a Series and a DataFrame, a function like `rolling_corr` will compute the correlation of the Series (`spx_rets`, in this case) with each column in the DataFrame (see Figure 11-9 for the plot of the result):

```
In [262]: corr = returns.rolling(125, min_periods=100).corr(spx_rets)
```

```
In [263]: corr.plot()
```



Figure 11-9. Six-month return correlations to S&P 500



윈도우 이동 함수 > User-Defined Moving Window Functions

Moving Window Functions

User-Defined Moving Window Functions

The `apply` method on rolling and related methods provides a means to apply an array function of your own devising over a moving window. The only requirement is that the function produce a single value (a reduction) from each piece of the array. For example, while we can compute sample quantiles using `rolling(...).quantile(q)`, we might be interested in the percentile rank of a particular value over the sample. The `scipy.stats.percentileofscore` function does just this (see [Figure 11-10](#) for the resulting plot):

```
In [265]: from scipy.stats import percentileofscore  
  
In [266]: score_at_2percent = lambda x: percentileofscore(x, 0.02)  
  
In [267]: result = returns.AAPL.rolling(250).apply(score_at_2percent)  
  
In [268]: result.plot()
```

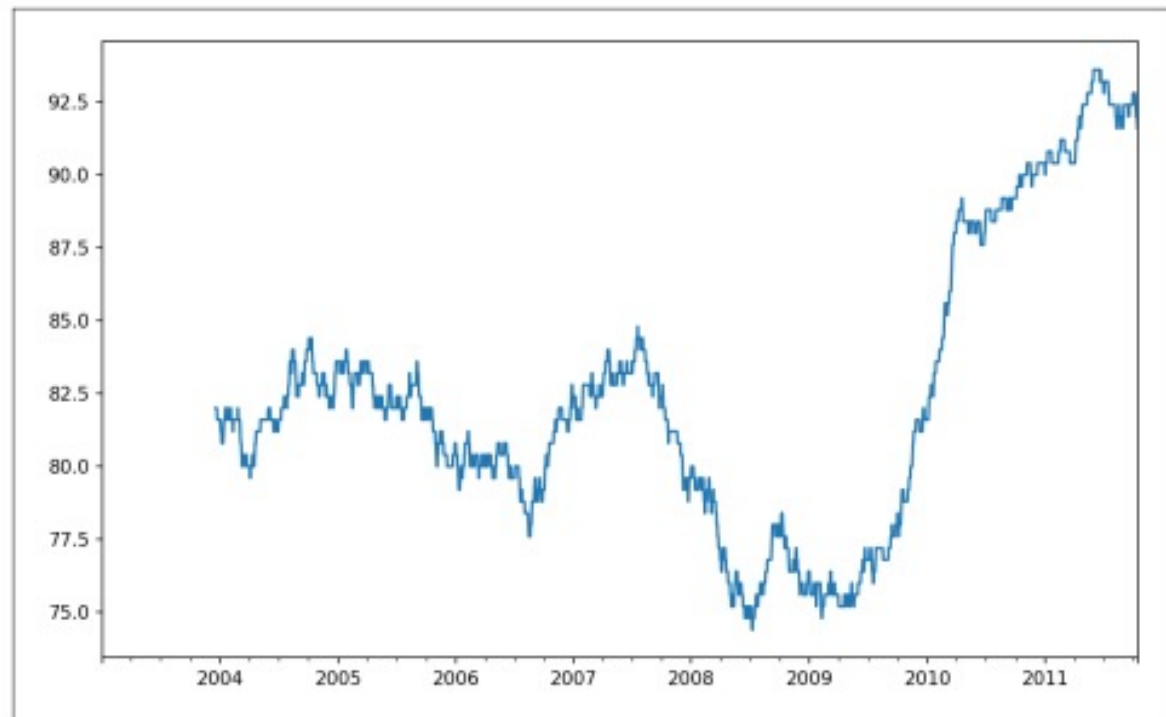


Figure 11-10. Percentile rank of 2% AAPL return over one-year window

If you don't have SciPy installed already, you can install it with conda or pip.

Q&A

질의응답 (5분)





다음 주 배울 내용

Contents for Next Week

판다스 정리

고급 판다스

판다스 활용 사례



프로젝트 진행 관련 참고자료

분석 기획에서의 데이터 사이언스 역량

- ▶ 분석 역량: 문제 영역에 대한 전문성, 수학·통계학적 지식
- ▶ 균형 잡힌 시각, 데이터·프로그래밍 기술 역량

분석 대상과 방법

방법 \ 대상	Known	Un-Known
Known	최적화 (Optimization)	통찰 (Insight)
Un-Known	솔루션 (Solution)	발견 (Discovery)

- ▶ 방법과 대상을 둘 다 알면 **최적화**, 방법을 모르면 방법을 찾아야 하므로 **솔루션** 둘 다 모르면 **발견** 대상을 모르면 **통찰**

분석 기획 방안

목표 시점 별 분석 기획 방안	
과제중심적 접근	장기적인 마스터 플랜
Speed & Test	Accuracy & Deploy
Quick & Win	Long Term View
문제 해결	문제 정의

- ▶ 과제 중심적 접근은 말그대로 과제를 정확도 보다 빠르게 해결하는데 중점을 둠

분석 기획시 고려사항

- ▶ 가용 데이터 : 분석의 기본이 되는 데이터 확보 및 파악
- ▶ 적절한 유스 케이스 : 기존에 잘 구현되어 활용되고 있는 유사 분석 시나리오 및 솔루션 최대한 활용
- ▶ 장애요소들에 대한 사전계획 수립 : 분석 수행 시 일어나는 문제에 대해 고려가 필요함 이행 저해요소 관리, 일회성 분석에 그치지 않고 조직의 **역량으로 내재화**

데이터 기반 의사결정의 필요성

- ▶ 경험과 감에 따른 직관적인 의사결정 → 데이터 기반의 의사결정
- ▶ 합리적 의사 결정을 가로막는 장애요소: **프레이밍 효과**, 고정 관념, 편향된 생각
↳ 동일 상황임에도 개인의 판단·결정이 달라짐

빅데이터 분석 방법론

- ▶ 분석 기획 → 데이터 준비 → **데이터 분석** → 시스템 구현 → 평가 & 전개
↳ 추가적인 데이터 확보가 필요한 경우 데이터 준비 단계로 다시 진행
- ▶ 분석 기획 단계는 **범위 설정** → **프로젝트 정의 & 계획** → **위험 식별 & 대응**으로 이루어짐



프로젝트 진행 관련 참고자료

분석 과제 관리를 위한 5가지 주요 요인

- ▶ 데이터 크기 & 데이터 복잡성 & 속도 & 분석 복잡성 & 정확성·정밀도
- ▶ 분석 복잡성에서 정확도와 복잡도는 trade off 관계가 존재한다. (정확도를 생각해 분석을 실행하면 복잡해질 것) & 정확성과 정밀도도 trade off 관계가 되는 경우가 많다.

분석 프로젝트 관리 방안 10가지

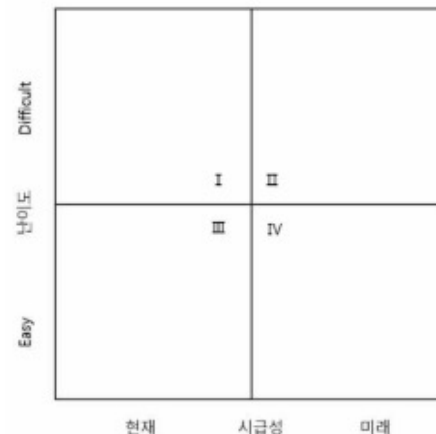
- ▶ 범위 시간 원가 품질 통합 조달 자원 리스크 의사소통 이해관계자
 - ↳ 외운다면 [범통이] [조리품] [시원 의자] 로 외우기
 - 범위통합이해관계자 조달리스크품질 시간원가 의사소통자원

분석 마스터 플랜 수립

- ▶ 전략적 중요도, 비즈니스 성과 및 ROI, 실행 용이성 등 기준을 고려해 적용 우선순위 결정
 - ↳ 전략적 중요도는 전략적 필요성과 시급성으로 이루어짐
 - 실행 용이성은 투자 용이성과 기술 용이성으로 이루어짐
- ▶ 업무 내재화 적용 수준, 분석 데이터 적용 수준, 기술 적용 수준을 고려해 Analytics 구현 로드맵 수립

ROI 관점 빅데이터 특징

- ▶ 투자 비용 요소(난이도) : 3V= Volume, Variety, Velocity
- ▶ 비즈니스 효과(시급성) : Value



사분면 분석

- ▶ 우선순위 : 시급성 기준으로는 III→IV→II (반시계 방향)
난이도 기준으로는 III→I→II (시계 방향)

분석 마스터 플랜 세부 이행 계획 수립

- ▶ 폭포수 방식도 있으나 반복적인 정련과정을 통하여 프로젝트의 완성도를 높이는 방식을 주로 사용
- ▶ 반복적인 분석체계라고 모든 단계를 반복하는 것이 아닌 모델링 단계를 중점으로 반복적으로 진행하고 세부적인 일정계획도 수립해야함



프로젝트 진행 관련 참고자료

분석을 위한 3가지 조직 구조



출처: <https://wikidocs.net/>

- ▶ 집중 구조는 말 그대로 분석 부서 하나가 따로 있는 것, 현업 업무부서와 분석 업무와 이원화 가능성이 높다. (이원화라는 말이 있으면 집중구조)
- ▶ 기능 구조는 별도의 분석 조직(DSCoE)이 없음
- ▶ 분산 구조는 분석 조직 인력들을 현업 부서로 직접 배치, 신속한 Action이 가능함.

분석 과제 관리 프로세스

- ▶ 과제 발굴 : 분석 idea 발굴 → 분석 과제 후보 제안 → 분석 과제 확정 →
- ▶ 과제 수행 : 결과 공유 & 개선 ← 분석 과제 진행 관리 ← 분석과제 실행 ← 팀 구성

※ 분석 과제 후보 제안, 결과 공유 단계에서만 Pool이라는 것에 관리함



프로젝트 진행 관련 참고자료

탐색적 자료분석(EDA): 데이터가 가지고있는 특성을 파악하기위해 시각화하여 분석하는 방식, **시각화하면 이상점을 식별하기 쉽다.**

▶ EDA의 4가지 주제: **저항성의 강조·잔차 계산·자료변수의 재표현·그래프를 통한 현시성**

기술통계: 모집단으로부터 표본을 추출하고 표본이 가지고 있는 정보를 하나의 숫자 또는 그래프의 형태로 표현 (우리가 흔히 생각하는 통계)

추측통계: 모집단으로부터 추출된 표본의 표본통계량으로 모집단을 **통계적으로 추론**

데이터마이닝 모델링

▶ 대표적인 고급 데이터 분석법이다.

↳ '시뮬레이션'도 고급 분석법 중 하나

▶ 지나치게 통계적 가설이나 유의성에 집착하지 말고 훈련 및 테스트 성능에 큰 편차가 없고 예상 성능을 만족하면 중단한다. 반드시 다양한 옵션을 주어야 하는 것이 아니다.

데이터 마트

▶ 데이터 웨어하우스와 사용자 사이의 중간층에 위치

▶ CRM관련 업무 중에서 핵심

요약변수

▶ 수집된 정보를 분석에 맞게 종합한 변수, **재활용성**이 높다.

파생변수

▶ 특정조건을 만족하거나 특정 함수에 의해 값을 만들어 의미를 부여한 변수

▶ **주관적일 수 있어 논리적 타당성**을 갖추어야한다.



프로젝트 진행 관련 참고자료

결측값(na) 처리 방법

- ▶ completes analysis: 결측값이 존재하는 레코드 삭제
 - ↳ 삭제하면 활용할 수 있는 변수의 수가 작아져 효율적이지 못함
- ▶ 평균 대체법: 결측값을 데이터의 평균으로 대체
- ▶ 다중 대체법: **대치** → **분석** → **결합** 단계로 진행

이상값

- ▶ 잘못 입력한 경우, 의도하지 않게 입력되었고 분석 목적에 맞지 않는 경우
 - ↳ 이런 경우는 웬만하면 삭제하는 것이 좋음
- ▶ 꼭 제거해야하는 것은 아니므로 적절한 판단 필요

이상값 인식 방법 3가지

- ▶ ESD: 평균으로부터 3 표준편차 떨어진 값
- ▶ 기하평균-2.5*표준편차 < data < 기하평균+2.5*표준편차를 벗어나는 값
- ▶ 사분위수 이용하기: $Q1 - 1.5 * (IQR) < data < Q3 + 1.5 * (IQR)$ 를 벗어나는 값
 - ↳ Q1: 데이터의 하위 25%에 해당하는 값(순서로 봤을 때)
 - Q3: 데이터의 상위 25%에 해당하는 값
 - $IQR = Q3 - Q1$
 - 보통 이러한 범위를 넘어가면 제거하지 않고 범위의 최대 최소값으로 바꾸어 활용
- ▶ 사기탐지, 의료, 부정사용방지 등등에 쓰일 수 있다.

통계자료 획득

- ▶ 전수조사: 전체를 다 조사하는 것 → 시간과 비용이 많이 소요.
- ▶ 표본조사: 일부만 추출하여 모집단(표본이 포함된 전체 집단)을 분석

표본 추출 방법

- ▶ 단순랜덤 추출법: 말 그대로 랜덤하게 표본을 뽑음
- ▶ 계통추출법: 번호를 랜덤하게 부여한 후 특정한 간격별로 추출
- ▶ 집락추출법: 군집을 나눈 후 군집별로 단순랜덤 추출
- ▶ 층화추출법: 계층을 고루 대표할 수 있도록 표본 추출

표본 오차 & 표본 편의

- ▶ 표본오차는 모집단을 대표할 수 있는 표본 단위들이 조사대상으로 추출되지 못함으로써 발생하는 오차
- ▶ 표본편의는 모수를 작게 또는 크게 할 때 추정하는 것과 같이 표본추출방법에서 기인하는 오차를 의미, **확률화**로 최소화하거나 없앨 수 있다.



프로젝트 진행 관련 참고자료

확률변수

- ▶ 이산형: 0이 아닌 확률값을 갖는 셀 수 있는 실수값
 - ↳ 이산형 확률 변수로는 베르누이·이항·기하·다항·포아송 분포가 있음
- ▶ 연속형: 확률이 함수형태로 주어져 있다고 생각하면 됨.
 - ↳ 균일분포·정규분포·지수분포·t-분포·카이제곱 분포·F-분포 가있다.
 - t-분포는 **평균**이 동일한지 알고자할 때 사용
 - 카이제곱 분포는 **모분산**에 대한 가설 검정에 사용
 - F-분포는 **분산의 동일성** 검정에 사용

조건부 확률

- ▶ $P(A|B) = \frac{P(A \cap B)}{P(B)}$ A와 B가 독립(배반)사건 이라면 $P(A \cap B) = P(A) * P(B)$ 이다.
 - ↳ 뒤에 있는 B가 분모로 감

추정: 표본으로부터 모수(모집단의 특징)를 추측하는 것

- ▶ 점추정: **모수가 특정한 값**일 것 이라고 추정하는 것
- ▶ 구간추정: **모수가 특정한 구간**에 있을 것이라고 선언하는 것
 - 추정량의 분포에 대한 전제가 주어져야 하고 구해진 구간 안에 모수가 있을 신뢰구간이 주어져야한다.

가설검정

- ▶ 귀무가설: 비교하는 값과 차이가 없음을 기본개념으로 하는 가설
 - ↳ ex.) '우리 반 평균키는 대한민국 남성 평균키보다 크다'라는 주장에 가설검정을 한다면 귀무가설은 '우리 반 평균키는 대한민국 남성 평균키와 차이가 없다.(같다.)'를 가설로 두는 것 (일반적으로 기각되는 것이 목표)
- ▶ 대립가설: 뚜렷한 증거가 있을 때 주장하는 가설 (채택되는 것이 목표)
- ▶ **p값(value):** 귀무가설이 사실일 때, 관측된 **검정통계량**이 **대립가설**을 지지하는 방향으로 나올 확률
 - ↳ **관찰된 표본으로부터 구한 통계량**
 - 우리가 내린 판정이 잘못되었을 실제 확률을 의미
- ▶ 유의수준: 귀무가설이 옳은데도 이를 기각(채택 안하는)하는 확률의 크기
- ▶ **제1종 오류:** 귀무가설이 옳은데 귀무가설을 기각하게 되는 오류
- ▶ **제2종 오류:** 귀무가설이 틀린데 귀무가설을 채택하는 오류



프로젝트 진행 관련 참고자료

회귀분석: 하나 또는 그 이상의 독립변수들이 종속변수에 미치는 영향을 추정하는 통계법
(함수의 개념과 비슷)

- ▶ p-값이 0.05보다 작으면 추정된 회귀식은 통계적으로 유의수준 5%이하에서 유의미하다고 봄
- ▶ 결정계수는 R-squared값으로 확인하면 되고 1에 가까울수록 설명력이 높다고 판단할 수 있다. 총 변동 중에서 회귀모형에 의하여 설명되는 변동이 차지하는 비율이다.

▶ 선형회귀분석의 가정

선형성	입력변수와 출력변수의 관계가 선형이다.
등분산성	오차의 분산이 입력변수와 무관하게 일정하다.
독립성	입력변수와 오차는 관련이 없다.
비상관성	오차들끼리 상관이 없다.
정상성	오차의 분포가 정규분포를 따른다. (Q-Q plot, Shapiro-Wilks test로 확인)

[정성(선)스럽게 비등한 독립] 으로 외우기 → 선형성 말고는 오차에 대한 내용이 주를 이룸.

↳ 정상성(선)형성스럽게 비상관성 등분산성한 독립성

- ▶ 데이터의 정규성은 Q-Q plot, Shapiro-Wilks test, 히스토그램을 사용해 확인 가능

단계적 변수선택

- ▶ **전진선택법:** 절편만 있는 상수모형으로부터 시작해 중요하다고 생각되는 변수를 차례로 추가
- ▶ **후진제거법:** 모든 변수를 포함한 모형에서 출발해 가장 적은 영향을 주는 변수부터 하나씩 제거
- ▶ **단계선택법:** 전진선택법으로 변수를 추가하는데 기존 변수가 영향을 받아 중요도가 약화되면 변수를 다시 제거하는 등 단계별로 추가,제거 여부를 검토하는 방법
- ▶ **최적회귀방정식**은 모든 후보 모형들에 대해 AIC, BIC를 계산하고 그 값이 최소가 되는 모형 선택 (AIC, BIC가 뭔지는 굳이 알 필요 없음)



프로젝트 진행 관련 참고자료

시계열 자료: 시간의 흐름에 따라 관찰된 값을 뜻함

정상시계열 (일반적으로 분산이 시점에 의존하지 않음을 나타냄)

- ▶ 모든 시점에 대해 일정한 평균과 분산을 가진다.
- ▶ 특정한 시차의 길이를 갖는 자기공분산을 측정하더라도 동일한 값을 갖는다.

비정상시계열을 정상시계열로 전환하는 방법

- ▶ 평균이 일정하지 않은 경우: 원시계열에 차분(현 시점에서 바로 전 시점의 자료값을 뺀)
- ▶ 계절성을 갖는 경우: 계절차분 사용
- ▶ 분산이 일정하지 않은 경우: 자연로그를 취함('변환' 이라고 함)

자기회귀 모형 (AR)

- ▶ 시계열 모델 중 자기 자신의 과거 값을 사용하여 설명하는 모형
- ▶ 백색 잡음의 현재값과 자기 자신의 과거값의 선형 가중합으로 이루어진 정상 확률 모형

분해시계열: 시계열에 영향을 주는 일반적인 요인을 시계열에서 분리해 분석하는 방법

- ↳ 경향(추세)요인, 계절요인, 순환요인, 불규칙요인으로 이루어짐
 - 경향은 말 그대로 자료가 오르거나 내리는 추세를 의미
 - 계절은 고정된 주기에 따라 자료가 변하는 경우
 - 순환은 경제적이나 자연적인 이유 없이 알려지지 않은 주기를 갖고 변화
 - 불규칙은 위 3가지로 설명할 수 없을 때

다차원척도법(MDS)

- ▶ 객체간 근접성을 시각화하는 통계기법
- ▶ 개체들을 2차원 또는 3차원 공간상에 점으로 표현하여 개체들 사이의 집단화를 시각적으로 표현하는 분석방법
- ▶ 계량적 MDS → 비율척도, 구간척도 데이터 활용
- ▶ 비계량적 MDS → 순서척도 데이터 활용

주성분분석(PCA)

- ▶ 여러 변수들을 상관관계를 이용해 소수의 주성분으로 차원을 축소하는 것
- ▶ Scree plot을 이용하는 경우에는 그래프의 기울기가 완만해져 거의 0에 다다른 지점에서 주성분의 개수를 구한다.
- ▶ 대략 85%의 분산설명력을 갖게끔 주성분의 수를 결정한다.

관측치 수 = 자유도(df) + 1

끝. 감사합니다.

수업 듣느라 수고하셨습니다.

